

Java 23 — Primitive Types in Patterns, instanceof, and switch (Preview)

Java Version: 23

Year: 2024

Feature: Primitive Types in Patterns, instanceof, and switch (Preview)

UniqueClassName: PrimitiveTypesPatterns

1. Definition

Primitive Types in Patterns, instanceof, and switch was introduced as a **Preview Feature in Java 23** through **JEP 455**.

It enhances pattern matching by allowing **primitive types to participate in type patterns and switch**, including conversions between primitive types and their wider/narrower types where permitted.

Before this feature, pattern matching was primarily designed around **reference types**:

```
if (obj instanceof Integer i) {  
    System.out.println(i);  
}
```

Java 23 extends the pattern-matching model so that primitive values can be tested directly against primitive type patterns.

For example:

```
int value = 100;  
  
if (value instanceof int i) {  
    System.out.println(i);  
}
```

The feature also enables primitive type patterns in switch:

```
int value = 100;  
  
switch (value) {  
    case int i -> System.out.println("Integer: " + i);  
}
```

Important: This was a **preview feature in Java 23**, so it requires preview compilation/execution options such as `--enable-preview`.

2. Why Was This Feature Introduced?

Pattern matching had been evolving in Java through several releases:

Java 14+ → instanceof pattern matching

Java 17+ → pattern matching improvements

Java 21 → Pattern Matching for switch finalized

Java 23 → Primitive Types in Patterns, instanceof, and switch

The goal was to make pattern matching more uniform across:

- Primitive types
- Reference types
- instanceof
- switch
- Type patterns
- Record patterns
- Numeric conversions

Instead of treating primitive values as a completely separate case, Java's pattern-matching system can reason about them using patterns.

3. Traditional Approach

Suppose we have:

```
int value = 100;
```

Traditionally, you might use:

```
if (value > 0) {  
    System.out.println("Positive");  
}
```

For objects, instanceof could perform type testing:

```
Object value = 100;
```

```
if (value instanceof Integer i) {  
    System.out.println("Integer: " + i);  
}
```

The new feature expands pattern matching so that **primitive types themselves can be used in patterns**.

4. Basic Syntax

A primitive type pattern has the general form:

`primitiveType variable`

For example:

`int i`

or:

`long l`

or:

`double d`

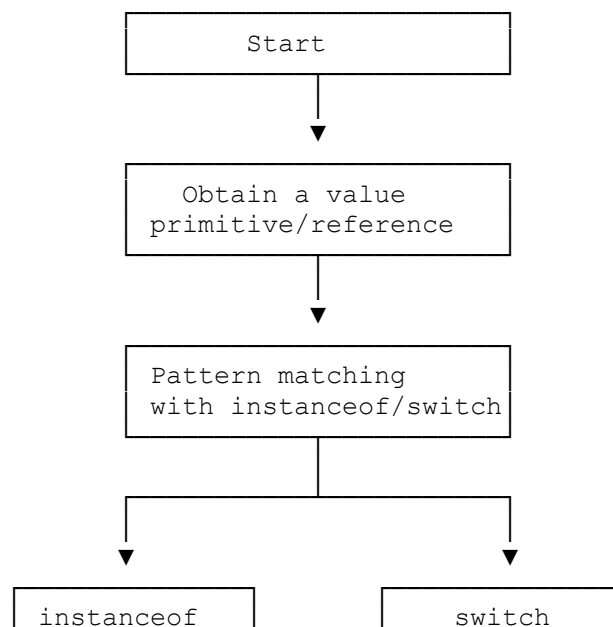
These can appear in pattern contexts such as:

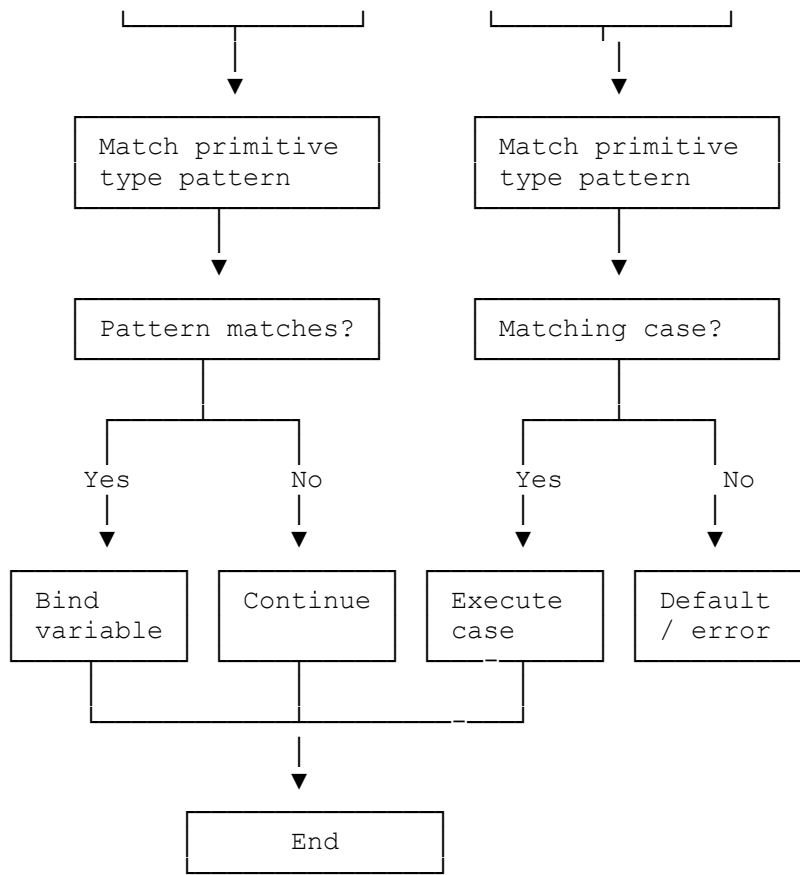
`instanceof`

and:

`switch`

5. Flowchart





6. Java Code Example — instanceof

Because this is a **Java 23 preview feature**, compile and run with preview enabled.

```
public class PrimitiveTypesPatterns {
    public static void main(String[] args) {
        int value = 100;

        if (value instanceof int i) {
            System.out.println("Primitive int matched.");
            System.out.println("Value = " + i);
        }
    }
}
```

Output

```
Primitive int matched.
Value = 100
```

The important part is:

value instanceof int i

Here:

- value → primitive int
 - int → primitive type pattern
 - i → pattern variable
 - i contains the matched value
-

7. Primitive Type Pattern with switch

Example:

```
public class PrimitiveTypesPatterns {  
    public static void main(String[] args) {  
        int value = 100;  
  
        String result = switch (value) {  
            case int i -> "Integer value: " + i;  
        };  
  
        System.out.println(result);  
    }  
}
```

Output

Integer value: 100

The pattern:

case int i

matches the primitive int value and binds it to:

i

8. Pattern with Wider Primitive Types

One of the important aspects of the feature is that patterns can participate in **primitive type conversions**.

For example:

```
public class PrimitiveTypesPatterns {  
    public static void main(String[] args) {  
        int value = 100;  
        switch (value) {  
            case long l ->  
                System.out.println("Matched as long: " + l);  
            default ->  
                System.out.println("No match");  
        }  
    }  
}
```

The int value can participate in a widening conversion to long.

Conceptually:

int
|
| widening conversion
▼
long

So the pattern can match the value as a long.

9. Multiple Primitive Patterns

The feature becomes particularly useful when different primitive types need to be handled.

Conceptually:

```
switch (value) {  
    case byte b    -> ...  
    case short s   -> ...  
    case int i      -> ...  
    case long l     -> ...  
    case float f    -> ...  
    case double d   -> ...  
}
```

However, **pattern applicability and dominance rules apply**. A broader/wider primitive pattern can make another pattern unreachable, so these cases cannot simply be combined arbitrarily.

10. Example with double

```
public class PrimitiveTypesPatterns {  
    public static void main(String[] args) {  
        double value = 45.75;  
  
        if (value instanceof double d) {  
            System.out.println("Double matched.");  
            System.out.println("Value = " + d);  
        }  
    }  
}
```

Output:

Double matched.
Value = 45.75

11. Example with long

```
public class PrimitiveTypesPatterns {  
    public static void main(String[] args) {  
        long value = 9_000_000_000L;  
  
        if (value instanceof long l) {  
            System.out.println("Long matched.");  
            System.out.println("Value = " + l);  
        }  
    }  
}
```

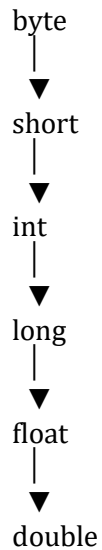
Output:

Long matched.
Value = 9000000000

12. Primitive Patterns and Numeric Conversion

A major idea behind this feature is that pattern matching can incorporate **numeric conversion**.

For example:



These are generally **widening primitive conversions**.

The pattern-matching implementation can use these conversions where the language rules permit them.

13. instanceof Before vs. Java 23 Preview

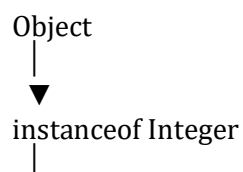
Before

instanceof was primarily used with reference types:

```
Object obj = Integer.valueOf(100);
```

```
if (obj instanceof Integer i) {  
    System.out.println(i);  
}
```

Flow:



▼
Integer i

Java 23 Preview

Primitive values can participate in primitive type patterns:

```
int value = 100;
```

```
if (value instanceof int i) {  
    System.out.println(i);  
}
```

Flow:

```
int value  
  |  
  ▼  
instanceof int  
  |  
  ▼  
int i
```

14. switch Before vs. Java 23 Preview

Traditional primitive switch:

```
int value = 10;
```

```
switch (value) {  
    case 10 -> System.out.println("Ten");  
    case 20 -> System.out.println("Twenty");  
    default -> System.out.println("Other");  
}
```

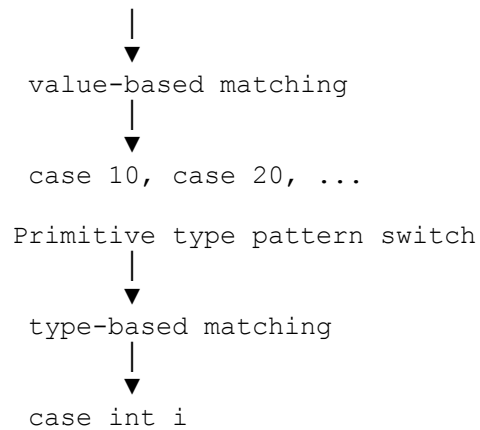
This switches on **constant values**.

With primitive type patterns, the switch can reason about the **type of the value and bind it to a variable**:

```
switch (value) {  
    case int i -> System.out.println("Integer: " + i);  
}
```

So there is a conceptual distinction:

Traditional switch



15. Complete Demonstration

```
public class PrimitiveTypesPatterns {  
    static void demonstrate(int value) {  
        System.out.println("Value: " + value);  
  
        if (value instanceof int i) {  
            System.out.println("instanceof matched int: " + i);  
        }  
  
        String result = switch (value) {  
            case int i -> "switch matched int: " + i;  
        };  
  
        System.out.println(result);  
    }  
  
    public static void main(String[] args) {  
        demonstrate(100);  
        demonstrate(500);  
        demonstrate(-25);  
    }  
}
```

Possible output:

```
Value: 100  
instanceof matched int: 100  
switch matched int: 100  
  
Value: 500  
instanceof matched int: 500  
switch matched int: 500
```

```
Value: -25
instanceof matched int: -25
switch matched int: -25
```

16. Compilation

Since this feature was a **preview feature in Java 23**, use:

```
javac --enable-preview --release 23 PrimitiveTypesPatterns.java
```

Run:

```
java --enable-preview PrimitiveTypesPatterns
```

17. Key Benefits

Benefit	Explanation
Primitive pattern matching	Primitive types can participate in patterns
instanceof enhancement	Primitive values can be tested using primitive type patterns
switch enhancement	Primitive type patterns can be used in switch cases
Numeric conversion	Permits appropriate primitive conversions during matching
Pattern variable	Matching values can be bound to variables
More uniform patterns	Primitive and reference pattern matching become more consistent
Better type-oriented code	Reduces some manual type/conversion logic

18. Important Restrictions

This feature was **Preview**, not a permanent Java SE feature in Java 23.

Therefore:

`--enable-preview`

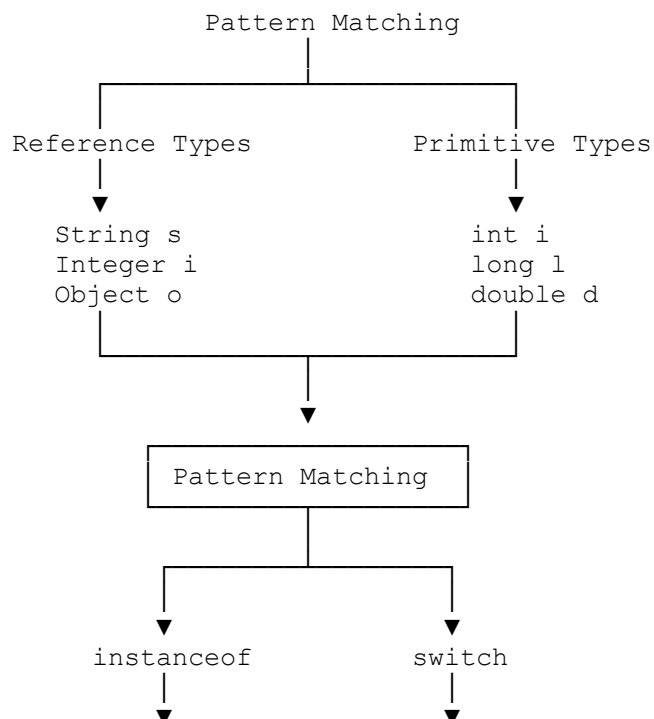
is required when compiling/running code that uses the preview feature.

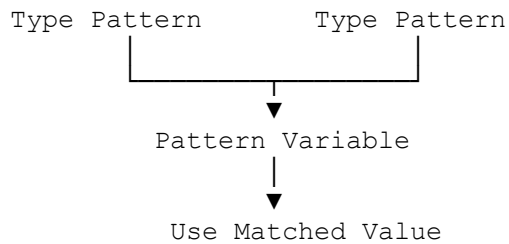
Also, primitive patterns are governed by Java's rules for:

- widening primitive conversions
- narrowing primitive conversions
- boxing/unboxing
- pattern compatibility
- pattern dominance
- exhaustiveness
- null
- floating-point values

So a primitive pattern should **not** be thought of simply as "instanceof for every primitive with no restrictions."

19. Conceptual Architecture





20. Summary

Java 23's Primitive Types in Patterns, instanceof, and switch (Preview) extends Java's pattern-matching capabilities from primarily reference types to **primitive types**.

The central idea is:

`primitiveValue instanceof primitiveType variable`

and:

```
switch (primitiveValue) {  
    case primitiveType variable -> ...  
}
```

For example:

```
int value = 100;
```

```
if (value instanceof int i) {  
    System.out.println(i);  
}
```

The feature makes Java's pattern-matching model more consistent and provides a foundation for handling **primitive type patterns and numeric conversions** directly within `instanceof` and `switch`.

Java 23 — Class-File API (Second Preview)

Java Version: 23

Year: 2024

Feature: Class-File API (Second Preview)

JEP: 466

UniqueClassName: ClassFileAPI

1. Definition

The **Class-File API** is a Java API for **parsing, generating, and transforming Java class files (.class files)** directly from Java code.

It provides a modern alternative to third-party bytecode libraries for many class-file manipulation tasks.

The API is located primarily in:

`java.lang.classfile`

It works with the structure of a JVM class file, including:

- Class metadata
- Fields
- Methods
- Attributes
- Bytecode instructions
- Constant-pool information
- Class-file versions

The API was introduced as a **preview feature in Java 22** and became the **second preview in Java 23**.

2. Why Class-File API?

Java applications sometimes need to generate or modify .class files dynamically.

Typical use cases include:

Frameworks

↓

Proxy generation

↓

Bytecode instrumentation

↓

Runtime code generation

↓

AOP
↓
Profilers
↓
Compilers
↓
Build tools

Historically, developers commonly used external libraries such as ASM for this type of work.

The Class-File API provides a **standard JDK API** for working with class files.

3. Main Idea

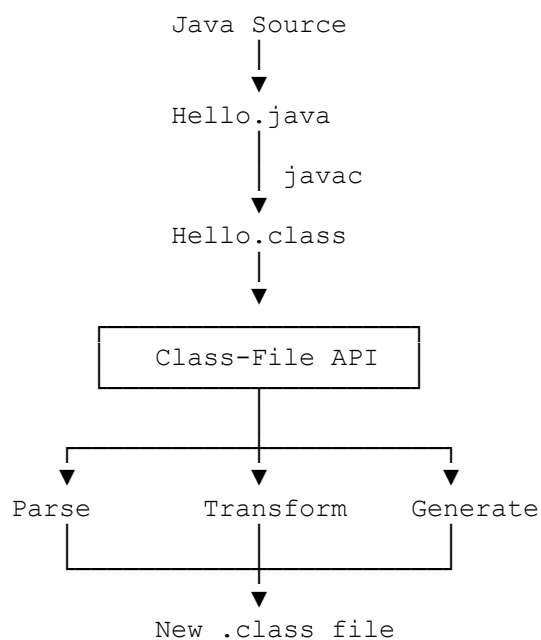
A Java source file:

Hello.java

is compiled into:

Hello.class

The Class-File API allows Java code to work directly with the .class representation.



4. Important Classes and Interfaces

The API contains several important types.

ClassFile

`java.lang.classfile.ClassFile`

It is the primary entry point for reading and generating class files.

For example:

`ClassFile.of()`

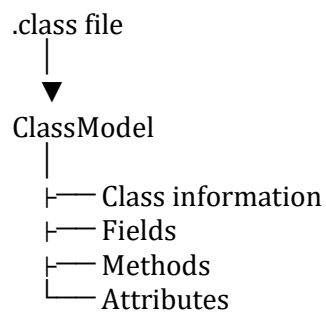
creates a `ClassFile` instance.

ClassModel

`java.lang.classfile.ClassModel`

Represents the parsed structure of a class file.

Conceptually:



ClassBuilder

`java.lang.classfile.ClassBuilder`

Used when constructing a new class file.

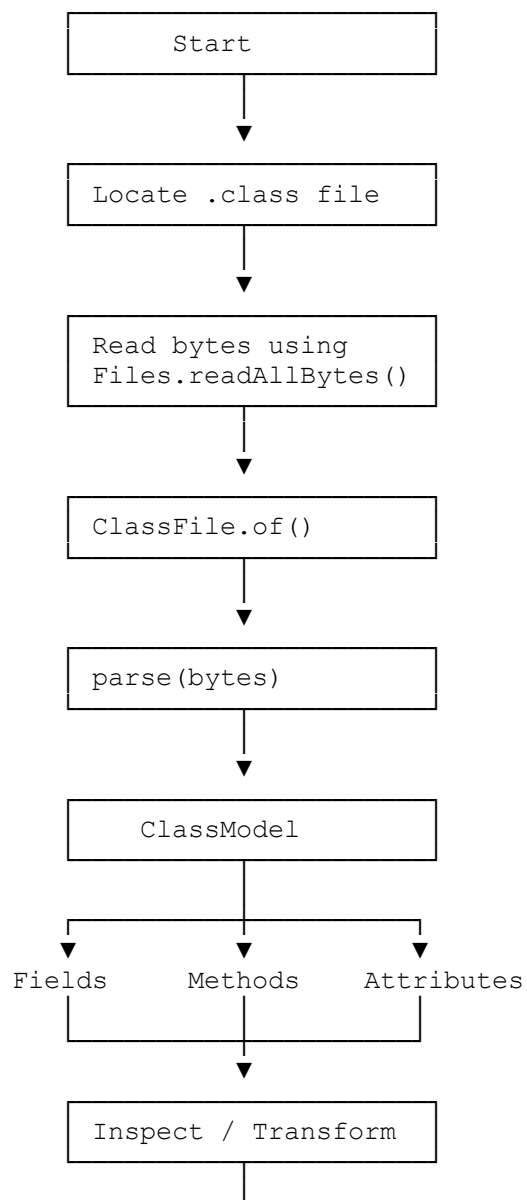
MethodBuilder

Used for constructing methods.

CodeBuilder

Used to construct JVM bytecode instructions inside a method.

5. Flowchart — Reading a Class File





End

6. Java Code Example — Parse a .class File

Suppose we have this class:

```
public class Hello {  
    public void sayHello() {  
        System.out.println("Hello");  
    }  
}
```

After compilation:

Hello.class

We can inspect it using the Class-File API.

```
import java.lang.classfile.ClassFile;  
import java.lang.classfile.ClassModel;  
  
import java.nio.file.Files;  
import java.nio.file.Path;  
  
public class ClassFileAPI {  
  
    public static void main(String[] args) throws Exception {  
  
        Path classFilePath = Path.of("Hello.class");  
  
        byte[] classBytes =  
            Files.readAllBytes(classFilePath);  
  
        ClassFile classFile = ClassFile.of();  
  
        ClassModel classModel =  
            classFile.parse(classBytes);  
  
        System.out.println(  
            "Class: " + classModel.thisClass().name()  
        );  
  
        System.out.println(  
            "Superclass: " + classModel.superclass()  
        );  
  
        System.out.println("Methods:");  
    }  
}
```

```

        classModel.methods().forEach(method ->
            System.out.println(
                "    " + method.methodName()
            )
        );
    }
}

```

Possible output

```

Class: Hello
Superclass: java/lang/Object
Methods:
    <init>
    sayHello

```

7. How the Example Works

Step 1 — Read the class file

```

byte[] classBytes =
    Files.readAllBytes(classFilePath);

```

The .class file is loaded into a byte array.

```

Hello.class
|
▼
byte[]

```

Step 2 — Create ClassFile

```

ClassFile classFile = ClassFile.of();

```

This provides the API's main operations.

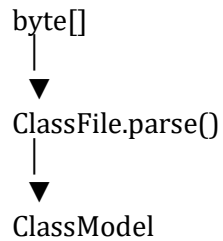
Step 3 — Parse

```

ClassModel classModel =
    classFile.parse(classBytes);

```

The binary class-file representation is converted into a structured model.



Step 4 — Inspect class information

`classModel.thisClass().name()`

This obtains the name of the class.

Step 5 — Inspect methods

`classModel.methods()`

returns the methods contained in the class.

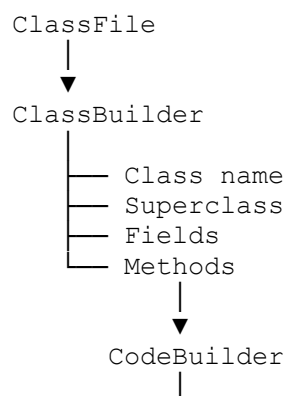
For example:

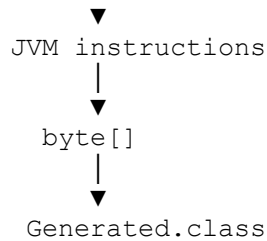
```
<init>  
sayHello
```

8. Generating a Class File

The Class-File API can also be used to **generate** class files.

The basic concept is:





A simplified example of class generation is:

```
import java.lang.classfile.ClassFile;

public class ClassFileAPI {

    public static void main(String[] args) throws Exception {

        ClassFile classFile = ClassFile.of();

        byte[] bytes = classFile.build(
            "example/GeneratedClass",
            classBuilder -> {
                // Class structure can be defined here.
            }
        );

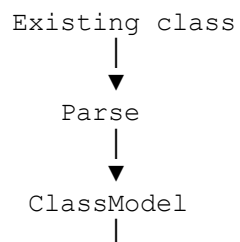
        System.out.println(
            "Generated class size: " +
            bytes.length +
            " bytes"
        );
    }
}
```

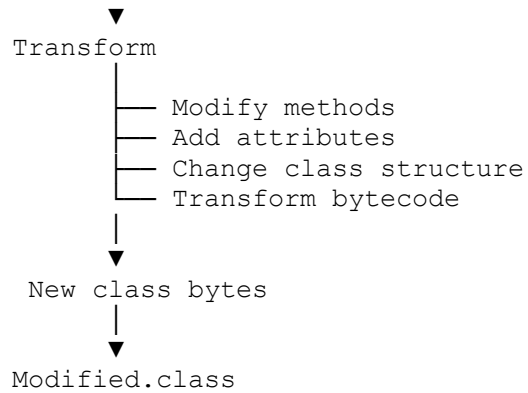
The exact builder operations depend on what class structure and bytecode you want to create.

9. Transforming a Class File

One of the most powerful uses is transformation.

For example:

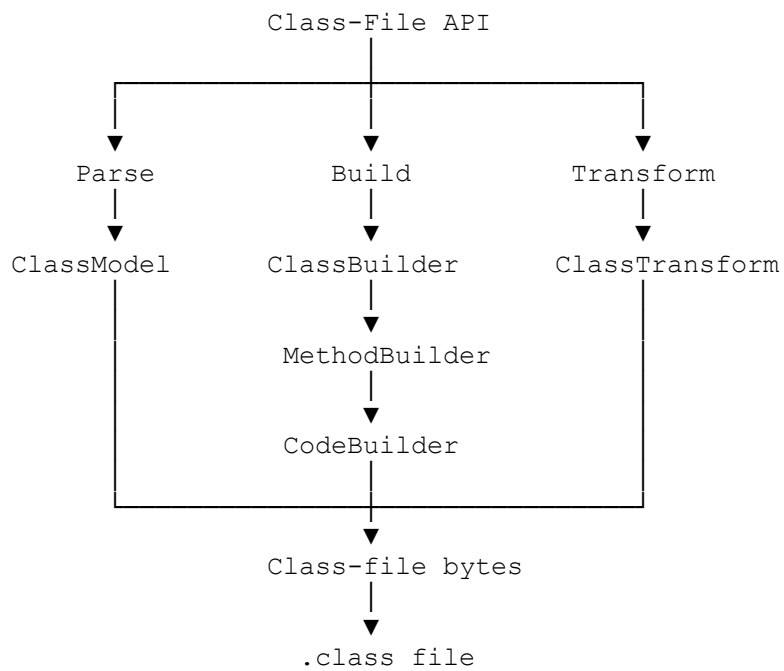




This can be useful for:

- instrumentation
- profiling
- monitoring
- framework implementation
- bytecode optimization
- dynamic class generation

10. Class-File API Architecture



11. Class-File API vs Reflection

These two APIs serve different purposes.

Feature	Reflection	Class-File API
Works with loaded classes	✓	Not primarily
Inspect fields/methods	✓	✓
Inspect .class binary	✗	✓
Generate .class files	✗	✓
Transform bytecode	✗	✓
Runtime metadata	✓	Limited/different
JVM bytecode instructions	✗	✓
Standard JDK API	✓	✓
Primary package	java.lang.reflect	java.lang.classfile

The important distinction is:

Reflection



Loaded Java class

Class-File API



Class-file representation

12. Class-File API vs ASM

Before the Class-File API, libraries such as ASM were commonly used for low-level class-file manipulation.

Conceptually:

Traditional approach

Java application



ASM
↓
.class manipulation

Java 23:

Java application
↓
Class-File API
↓
.class manipulation

The Class-File API is designed specifically for the JVM class-file format and is maintained as part of the JDK.

13. Preview Status

In Java 23, this API was **not yet finalized**.

Its evolution was:

Java 22
↓
Class-File API
First Preview
↓
Java 23
↓
Class-File API
Second Preview

Therefore, Java 23 code using this preview API requires preview support.

Compile:

```
javac --enable-preview --release 23 ClassFileAPI.java
```

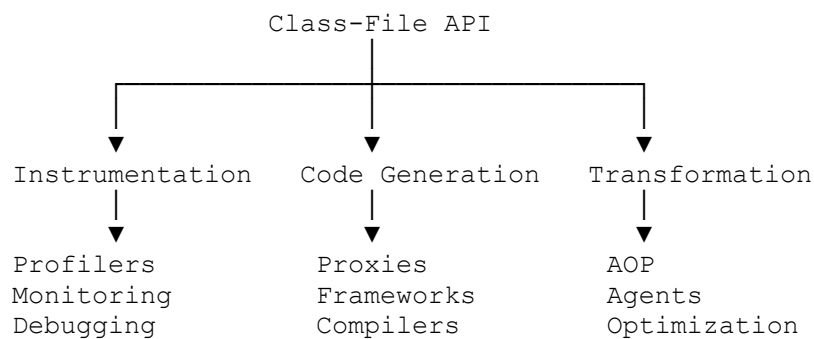
Run:

```
java --enable-preview ClassFileAPI
```

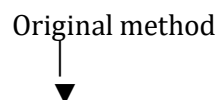
14. Key Benefits

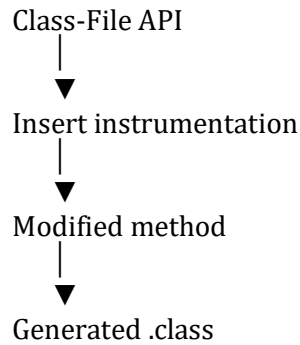
Benefit	Description
Standard JDK API	No external bytecode library is required for supported use cases
Parse class files	Read .class files programmatically
Generate classes	Create JVM class-file structures
Transform classes	Modify class-file structures and bytecode
Bytecode access	Work with JVM instructions
Type-safe API	Provides structured Java representations
Framework support	Useful for frameworks and runtime infrastructure
Instrumentation	Useful for monitoring and profiling
Modern design	Designed alongside modern JVM/class-file evolution

15. Real-World Use Cases



For example, an AOP framework could conceptually perform:





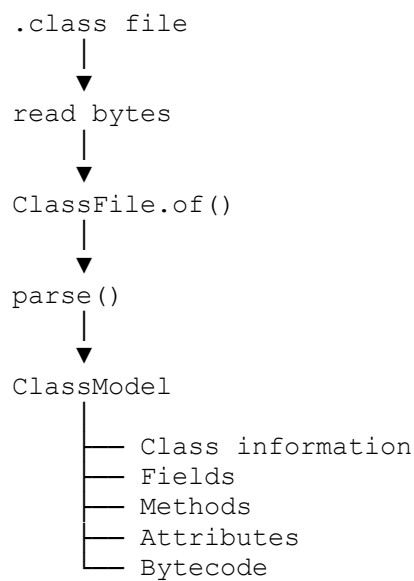
16. Summary

Class-File API (Second Preview) in Java 23 provides a standard JDK mechanism for **reading, generating, and transforming JVM class files**.

The central API is:

`java.lang.classfile.ClassFile`

The basic parsing workflow is:



The most important conceptual difference is:

Reflection works primarily with already-loaded Java classes, whereas the Class-File API works directly with the JVM class-file representation.

Because **Java 23's Class-File API was a second preview**, use `--enable-preview` when compiling and running Java 23 code that depends on it.

Java 23 — Markdown Documentation Comments

Java Version: 23

Year: 2024

Feature: Markdown Documentation Comments

JEP: 467

Status: Final

UniqueClassName: MarkdownDocumentationComments

1. Definition

Markdown Documentation Comments is a Java 23 feature that enhances **Javadoc comments** by allowing developers to use **Markdown syntax directly inside documentation comments**.

Before Java 23, Javadoc primarily used HTML-style markup:

```
/**
 * Returns the <b>maximum</b> value.
 *
 * @param a first value
 * @param b second value
 * @return maximum value
 */
```

With Java 23, documentation comments can use familiar Markdown syntax:

```
/// Returns the maximum value.
///
/// - `a` is the first value
/// - `b` is the second value.
///
/// @param a first value
/// @param b second value
/// @return the maximum value
```

The feature is specified by **JEP 467: Markdown Documentation Comments** and was finalized in Java 23.

2. Why Was Markdown Added to Javadoc?

Traditional Javadoc comments require developers to mix documentation with HTML:

```
/**
 * <h2>Calculator</h2>
 *
 * <p>This class provides <b>basic</b> mathematical operations.</p>
 *
 * <ul>
 *   <li>Addition</li>
 *   <li>Subtraction</li>
 * </ul>
 */
```

Markdown is generally easier to read in source code:

```
/**
 * ## Calculator
 *
 * This class provides basic mathematical operations.
 *
 * - Addition
 * - Subtraction
 */
```

The goal is to make documentation:

- Easier to write
 - Easier to read in source code
 - More natural for developers
 - Better suited to modern documentation practices
-

3. Basic Syntax

Java 23 supports Markdown within traditional Javadoc comments:

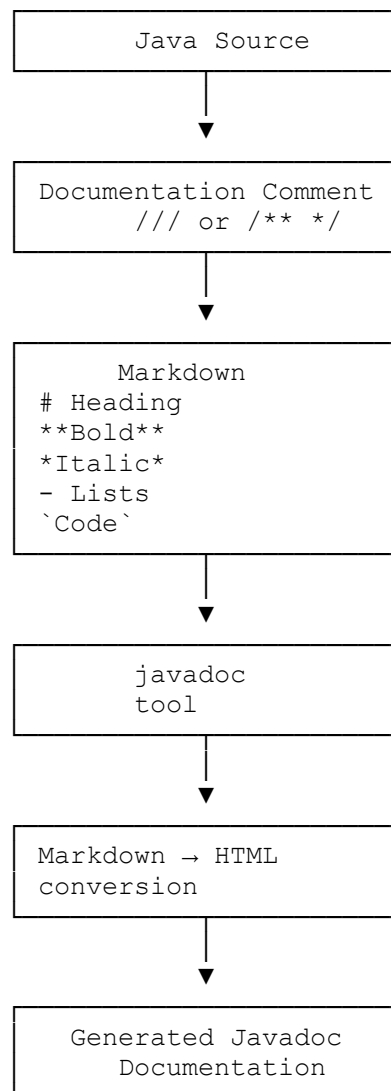
```
/**
 * # Heading
 *
 * This is bold text.
 *
 * This is italic text.
 *
 * - Item 1
 * - Item 2
 *
 * `code`
 */
```

It also introduces support for **documentation comments using `///`**.

Example:

```
/// # Calculator  
///  
/// Provides basic mathematical operations.
```

4. Flowchart



5. Java Code Example

Here is a simple example using the new Markdown documentation style.

```
public class MarkdownDocumentationComments {

    /// # Calculator
    ///
    /// A simple calculator that provides basic
    /// mathematical operations.
    ///
    /// ## Supported Operations
    ///
    /// - Addition
    /// - Subtraction
    /// - Multiplication
    /// - Division
    ///
    /// Example:
    ///
    /// `int result = add(10, 20);`
    ///
    /// @param a first number
    /// @param b second number
    /// @return the sum of `a` and `b`
    public static int add(int a, int b) {
        return a + b;
    }

    public static void main(String[] args) {

        int result = add(10, 20);

        System.out.println("Result = " + result);
    }
}
```

Output

Result = 30

The important part is that the documentation is written using:

```
/// # Calculator
///
/// A simple calculator that provides basic
/// mathematical operations.
```

6. Traditional Javadoc vs Markdown Javadoc

Traditional HTML-style Javadoc

```
/**
 * <h2>Calculator</h2>
 *
 * <p>A simple <b>calculator</b>.</p>
 *
 * <ul>
 *   <li>Addition</li>
 *   <li>Subtraction</li>
 * </ul>
 */
```

Java 23 Markdown style

```
/**
 * ## Calculator
 *
 * A simple calculator.
 *
 * - Addition
 * - Subtraction
 */
```

Markdown makes the source documentation considerably easier to read.

7. /// Documentation Comments

One of the most visible parts of JEP 467 is the ability to use `///` for documentation comments.

Example:

```
/// Calculates the square of a number.
///
/// The formula is:
///
///  $n \times n$ 
public static int square(int n) {
    return n * n;
}
```

This avoids repeatedly writing:

```
/**
 *
 */
```

and:

*

for every line.

8. Markdown Headings

You can use Markdown headings.

```
/**
 * # Calculator
 *
 * ## Addition
 *
 * Adds two numbers.
 *
 * ## Multiplication
 *
 * Multiplies two numbers.
 */
```

Conceptually:

```
# Calculator
├── ## Addition
└── ## Multiplication
```

9. Bold and Italic Text

Markdown allows:

```
/**
 * This is important information.
 *
 * This is optional information.
 */
```

Rendered documentation will interpret:

```
important
  ↓
bold

optional
  ↓
```

italic

10. Lists

Markdown lists can be used naturally.

```
/**
 * ## Features
 *
 * - Fast
 * - Simple
 * - Type-safe
 * - Easy to maintain
 */
```

This is easier to read than an equivalent HTML list:

```
/**
 * <ul>
 *   <li>Fast</li>
 *   <li>Simple</li>
 *   <li>Type-safe</li>
 *   <li>Easy to maintain</li>
 * </ul>
 */
```

11. Inline Code

Use backticks for inline code:

```
/**
 * Call `calculate()` to perform the calculation.
 */
```

This is particularly useful when documenting:

- Class names
 - Method names
 - Variables
 - Java expressions
 - Commands
 - Configuration values
-

12. Code Blocks

Markdown fenced code blocks can be used in documentation.

```
/**
 * ## Example
 *
 * ```java
 * int result = add(10, 20);
 * System.out.println(result);
 * ```
 */
```

The documentation can therefore contain source-code examples without requiring HTML `<pre>` and `<code>` tags.

13. Links

Markdown-style links can also be used.

Conceptually:

```
/**
 * See [Java Documentation] (https://docs.oracle.com/)
 * for more information.
 */
```

Javadoc processes the Markdown and generates the appropriate HTML documentation.

14. Markdown + Javadoc Tags

Markdown does **not replace Javadoc tags** such as:

@param
@return
@throws
@see
@since
@deprecated

They can be combined.

Example:

```

/**
 * # Calculator
 *
 * Calculates the average of two numbers.
 *
 * @param a first number
 * @param b second number
 * @return the calculated average
 * @throws ArithmeticException if calculation is invalid
 */
public static double average(int a, int b) {
    return (a + b) / 2.0;
}

```

So the model is:

Markdown

+

Javadoc tags

↓

Rich Java documentation

15. Complete Example

```

public class MarkdownDocumentationComments {

    /// # Mathematical Utilities
    ///
    /// Provides a collection of basic mathematical
    /// operations.
    ///
    /// ## Operations
    ///
    /// - Addition
    /// - Subtraction
    /// - Multiplication
    /// - Division

    /// Adds two integers.
    ///
    /// Example:
    ///
    /// ```java
    /// int result = add(10, 20);
    /// ```
    ///
    /// @param a first integer
    /// @param b second integer
    /// @return the sum of `a` and `b`
    public static int add(int a, int b) {
        return a + b;
    }
}

```

```

    }

    /// Multiplies two integers.
    ///
    /// The operation uses the standard multiplication
    /// operator `*`.
    ///
    /// @param a first integer
    /// @param b second integer
    /// @return the product of `a` and `b`
    public static int multiply(int a, int b) {
        return a * b;
    }

    public static void main(String[] args) {

        int sum = add(10, 20);
        int product = multiply(10, 20);

        System.out.println("Sum = " + sum);
        System.out.println("Product = " + product);
    }
}

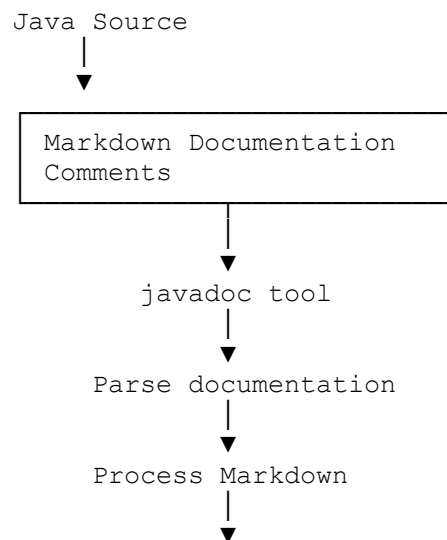
```

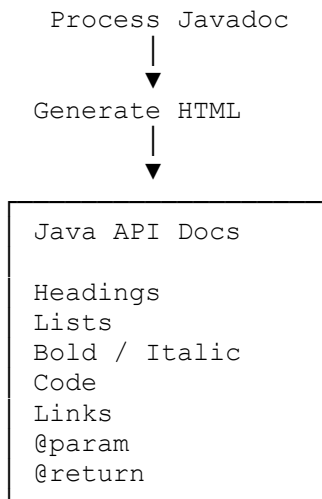
Output

Sum = 30
Product = 200

16. How Javadoc Processes Markdown

The overall process is:





17. Running javadoc

Suppose the source file is:

MarkdownDocumentationComments.java

You can generate documentation using:

javadoc MarkdownDocumentationComments.java

The Javadoc tool generates HTML documentation.

Unlike several other Java 23 preview features, **Markdown Documentation Comments** was **finalized in Java 23**, so it does **not** require:

--enable-preview

18. Key Benefits

Benefit	Description
Markdown support	Documentation can use familiar Markdown syntax
Better readability	Source comments are easier to read
/// syntax	Concise documentation comments
Lists	Markdown lists are easy to create

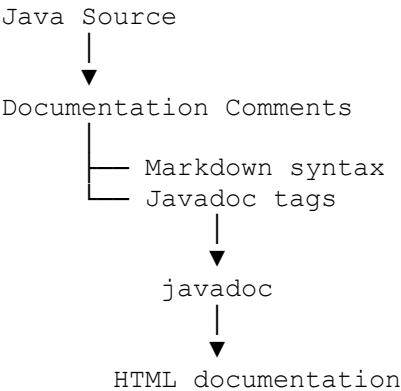
Headings	#, ##, etc. can structure documentation
Formatting	Bold, italic, code, etc.
Code examples	Markdown code blocks can be used
Javadoc compatibility	Existing Javadoc tags remain available
Standard JDK	No external Markdown library is required for Javadoc processing
Better developer experience	Documentation is closer to modern Markdown-based documentation

19. Important Point

Markdown documentation **does not mean that Java source comments become general-purpose Markdown files.**

The Markdown is specifically interpreted as part of the **Javadoc documentation-generation process.**

Think of it as:



20. Java 23 Feature Summary

Property	Value
Java Version	23
Year	2024

Feature	Markdown Documentation Comments
JEP	JEP 467
Status	Final
UniqueClassName	MarkdownDocumentationComments
Main tool	javadoc
Traditional syntax	<code>/** ... */</code>
New concise syntax	<code>/// ...</code>
Preview flag required?	✗ No

In one sentence

Markdown Documentation Comments in Java 23 allow developers to write Javadoc using familiar Markdown syntax, including headings, lists, emphasis, code, and links, while retaining traditional Javadoc tags such as `@param` and `@return`.

Java 23 — Vector API (Eighth Incubator)

Java Version: 23

Year: 2024

Feature: Vector API (Eighth Incubator)

JEP: 469

Status: Eighth Incubator

UniqueClassName: VectorAPI

1. Definition

The **Vector API** is an incubating Java API for expressing **vector computations** that can take advantage of modern CPU **SIMD (Single Instruction, Multiple Data)** hardware.

Instead of processing values one at a time:

```
a1 + b1
a2 + b2
a3 + b3
```

$a_4 + b_4$

vector processing can perform multiple operations together:

$$\begin{array}{r} [a_1, a_2, a_3, a_4] \\ + \\ [b_1, b_2, b_3, b_4] \\ \downarrow \\ [c_1, c_2, c_3, c_4] \end{array}$$

The Java Vector API is primarily located in:

`jdk.incubator.vector`

In Java 23, it was the **eighth incubation** of the API.

2. What Is SIMD?

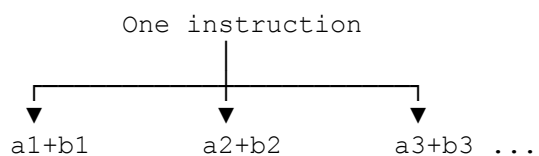
SIMD means:

Single Instruction, Multiple Data

A normal scalar operation might process one value:

1 operation
↓
▼
 $a + b$

SIMD can process multiple values with one vector instruction:



This can be especially useful for computationally intensive workloads.

3. Why Vector API?

Consider adding two large arrays.

Scalar approach

```
for (int i = 0; i < length; i++) {
    result[i] = a[i] + b[i];
}
```

Conceptually:

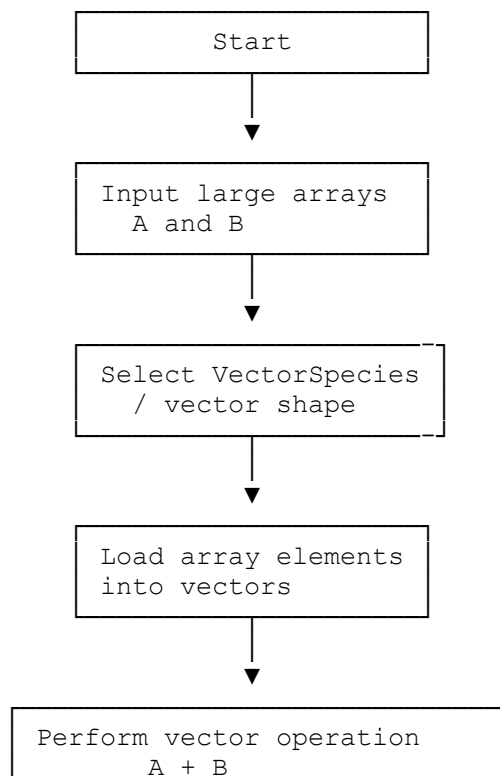
```
a[0] + b[0]
a[1] + b[1]
a[2] + b[2]
a[3] + b[3]
...
```

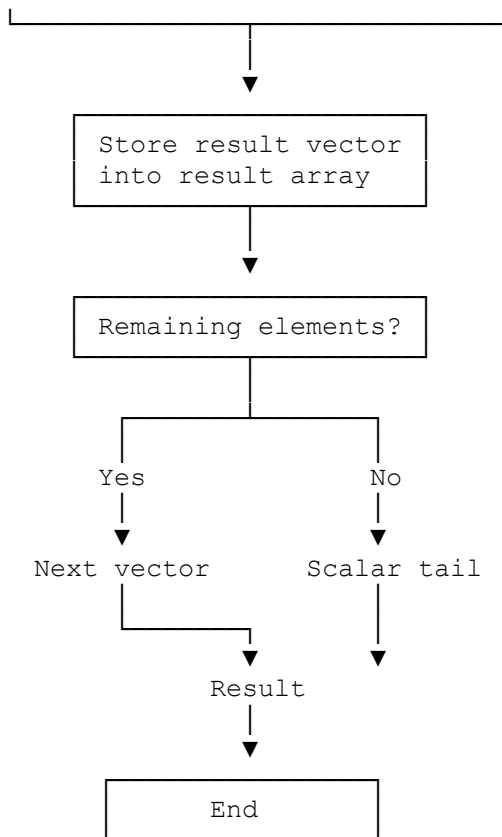
The Vector API allows the programmer to express the computation as vector operations:

```
[a0 a1 a2 a3]
      +
[b0 b1 b2 b3]
      ↓
[c0 c1 c2 c3]
```

The JVM can then map appropriate vector operations onto available CPU SIMD instructions.

4. Flowchart





5. Important Vector API Classes

The API is in:

`jdk.incubator.vector`

Some important types include:

VectorSpecies<E>

Represents a vector shape/species.

Example:

`IntVector.SPECIES_PREFERRED`

This asks the JVM for a preferred vector size for int.

IntVector

Represents a vector containing int values.

IntVector

FloatVector

Represents a vector containing float values.

FloatVector

DoubleVector

Represents a vector containing double values.

DoubleVector

LongVector

Represents a vector containing long values.

LongVector

6. Basic Vector Operation

Suppose:

A = [1, 2, 3, 4]

B = [10, 20, 30, 40]

Vector addition:

[1,	2,	3,	4]
+	+	+	+
[10,	20,	30,	40]
↓	↓	↓	↓
[11,	22,	33,	44]

Instead of explicitly writing four additions, the Vector API expresses the operation as one vector addition.

7. Java Code Example — Vector Addition

Because the API is an **incubator module**, compile and run using the incubator module.

```
import jdk.incubator.vector.IntVector;
import jdk.incubator.vector.VectorSpecies;

public class VectorAPI {

    private static final VectorSpecies<Integer> SPECIES =
        IntVector.SPECIES_PREFERRED;

    public static void main(String[] args) {

        int[] a = {
            1, 2, 3, 4,
            5, 6, 7, 8
        };

        int[] b = {
            10, 20, 30, 40,
            50, 60, 70, 80
        };

        int[] result = new int[a.length];

        for (int i = 0;
             i < a.length;
             i += SPECIES.length()) {

            if (i + SPECIES.length() <= a.length) {

                IntVector va =
                    IntVector.fromArray(SPECIES, a, i);

                IntVector vb =
                    IntVector.fromArray(SPECIES, b, i);

                IntVector vc =
                    va.add(vb);

                vc.intoArray(result, i);
            }
        }

        for (int value : result) {
            System.out.print(value + " ");
        }
    }
}
```

Output

11 22 33 44 55 66 77 88

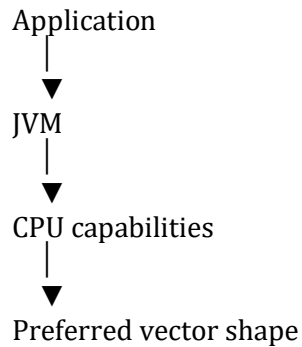
8. How the Code Works

Step 1 — Select preferred vector species

```
private static final VectorSpecies<Integer> SPECIES =  
    IntVector.SPECIES_PREFERRED;
```

This asks the JVM for a vector shape appropriate for the current platform.

Conceptually:



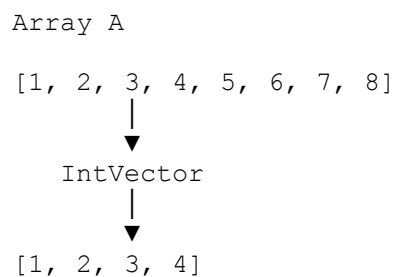
This helps make vectorized code portable across different hardware.

Step 2 — Load data

```
IntVector va =  
    IntVector.fromArray(SPECIES, a, i);
```

This loads multiple int values from the array into a vector.

For example:



The actual number of lanes depends on the selected species and hardware.

Step 3 — Perform vector addition

```
IntVector vc = va.add(vb);
```

Conceptually:

```
va = [1, 2, 3, 4]
vb = [10, 20, 30, 40]
```

```
    va
    +
    vb
    ↓
vc = [11, 22, 33, 44]
```

Step 4 — Store the vector

```
vc.toArray(result, i);
```

The vector values are written back to the result array.

```
IntVector
  ↓
[11, 22, 33, 44]
  ↓
result[]
```

9. Vector Processing vs Scalar Processing

Scalar

```
for (int i = 0; i < a.length; i++) {
    result[i] = a[i] + b[i];
}
```

Conceptually:

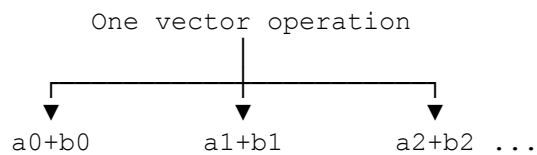
Iteration 1 $\rightarrow a_0 + b_0$
Iteration 2 $\rightarrow a_1 + b_1$
Iteration 3 $\rightarrow a_2 + b_2$
Iteration 4 $\rightarrow a_3 + b_3$

Vector

```
IntVector va = IntVector.fromArray(SPECIES, a, i);  
IntVector vb = IntVector.fromArray(SPECIES, b, i);
```

```
IntVector vc = va.add(vb);
```

Conceptually:



10. Vector Species

A **VectorSpecies** describes the shape and element type of a vector.

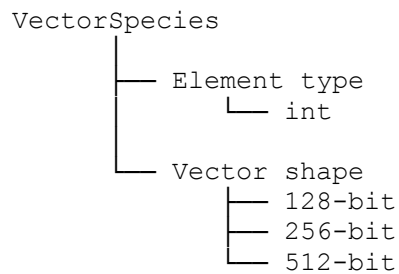
For example:

```
IntVector.SPECIES_PREFERRED
```

Other concepts include fixed vector shapes such as:

```
IntVector.SPECIES_128  
IntVector.SPECIES_256  
IntVector.SPECIES_512
```

Conceptually:



The preferred species is generally useful when you want the JVM to choose an appropriate vector size for the current platform.

11. Vector Lanes

A vector consists of multiple **lanes**.

For example, conceptually:

10	20	30	40
↑	↑	↑	↑
lane	lane	lane	lane
0	1	2	3

Each lane contains one element.

For an `IntVector`, each lane contains an `int`.

12. Other Vector Operations

The API supports many mathematical and logical operations.

Addition

`va.add(vb)`

Subtraction

`va.sub(vb)`

Multiplication

`va.mul(vb)`

Division

`va.div(vb)`

Minimum

`va.min(vb)`

Maximum

va.max(vb)

Conceptually:

Vector API

- add()
- sub()
- mul()
- div()
- min()
- max()
- and()
- or()
- xor()

13. Example — Vector Multiplication

```
import jdk.incubator.vector.DoubleVector;
import jdk.incubator.vector.VectorSpecies;

public class VectorAPI {

    public static void main(String[] args) {

        VectorSpecies<Double> species =
            DoubleVector.SPECIES_PREFERRED;

        double[] a = {1.0, 2.0, 3.0, 4.0};
        double[] b = {10.0, 20.0, 30.0, 40.0};

        double[] result = new double[a.length];

        for (int i = 0;
            i < species.length();
            i += species.length()) {

            DoubleVector va =
                DoubleVector.fromArray(species, a, i);

            DoubleVector vb =
                DoubleVector.fromArray(species, b, i);

            DoubleVector vc =
                va.mul(vb);

            vc.intoArray(result, i);
        }

        for (double value : result) {
            System.out.print(value + " ");
        }
    }
}
```

```

    }
  }
}

```

Conceptually:

```

[1, 2, 3, 4]
  ×
[10, 20, 30, 40]
  ↓
[10, 40, 90, 160]

```

14. Vector Mask

The Vector API also supports **masks**.

A mask determines which lanes participate in an operation.

For example:

Vector:

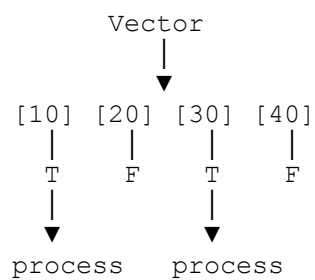
[10 20 30 40]

Mask:

[T F T F]

Only selected lanes can be operated on.

Conceptually:

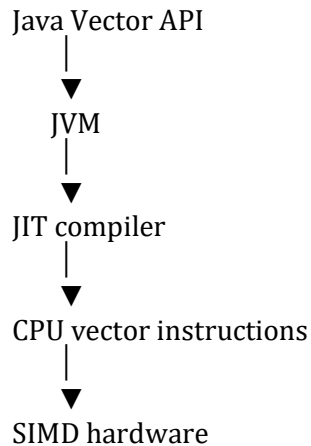


Masks are especially useful for conditional vector operations.

15. Vector API and CPU

The JVM can translate vector operations into suitable hardware instructions when supported.

Conceptually:



Depending on the processor, the generated machine code may use hardware vector instruction sets.

The Java programmer does **not** need to write processor-specific assembly code.

16. Important Advantage

A major design goal is **platform independence**.

Instead of writing:

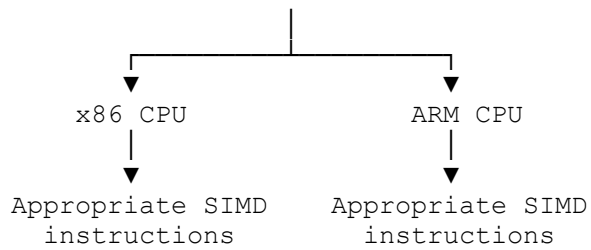
```
x86-specific SIMD code  
OR  
ARM-specific SIMD code
```

you can write:

```
IntVector  
DoubleVector  
FloatVector  
LongVector
```

and allow the JVM/JIT to map those operations to appropriate hardware.

Java Vector API



17. Compilation and Execution

Because the Vector API was an **incubator API in Java 23**, you need to explicitly add the incubator module.

Compile:

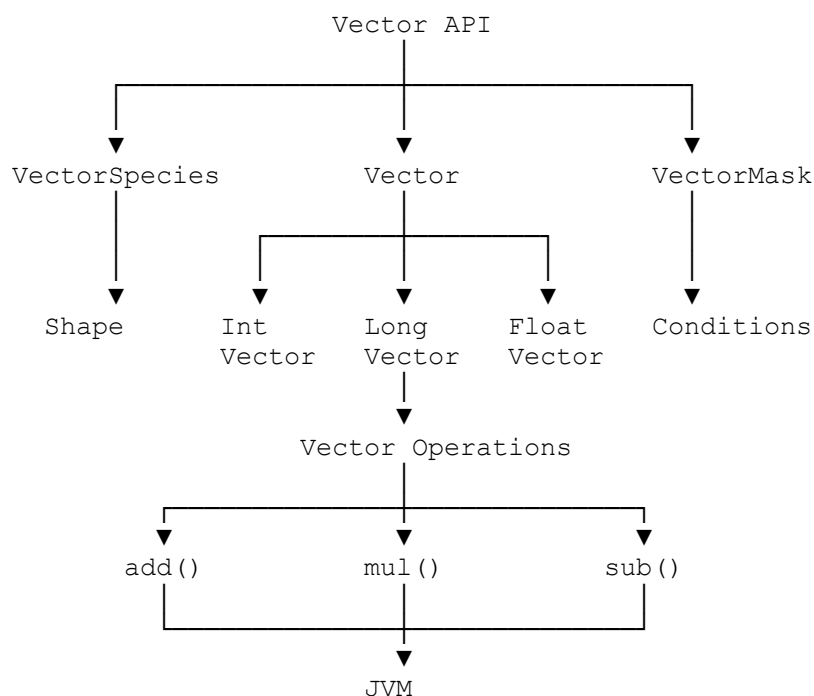
```
javac --add-modules jdk.incubator.vector VectorAPI.java
```

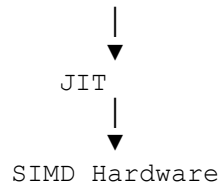
Run:

```
java --add-modules jdk.incubator.vector VectorAPI
```

You may see an informational warning indicating that an incubating module is being used.

18. Vector API Architecture





19. Real-World Applications

The Vector API can be valuable in workloads involving large amounts of numerical data.

```
Vector API
├── Scientific computing
├── Machine learning
├── Image processing
├── Audio processing
├── Signal processing
├── Financial calculations
├── Cryptography-related computations
├── Physics simulations
└── Data analytics
```

For example, image processing often performs the same operation on many pixels:

```
Pixel 1 → operation
Pixel 2 → operation
Pixel 3 → operation
Pixel 4 → operation
...
```

Vectorization can process multiple pixel values together.

20. Vector API vs Parallel Streams

These approaches solve different problems.

Feature	Vector API	Parallel Streams
Main goal	Data-level parallelism	Task/data parallelism
Level	CPU/SIMD	Multiple threads
Uses SIMD	✓	Not directly
Uses multiple threads	Not inherently	✓

Low-level control	Higher	Lower
Numerical workloads	Excellent candidate	Possible
CPU vector instructions	Can exploit	JVM may optimize independently
API	jdk.incubator.vector	java.util.stream

They can sometimes complement each other, but vectorization and multithreading are **different levels of parallelism**.

21. Scalar + Vector Processing

Real arrays are not always an exact multiple of the vector length.

For example:

Array length = 10
Vector length = 4

Processing might look like:

Vector 1 → elements 0–3
Vector 2 → elements 4–7
Scalar → elements 8–9

Flow:

```

10 elements
├── [0 1 2 3] → Vector
├── [4 5 6 7] → Vector
└── [8 9]     → Remaining elements

```

This remaining portion is commonly called the **tail**.

The Vector API provides mechanisms such as masks to handle such cases safely.

22. Key Benefits

Benefit	Explanation
SIMD	Performs operations on multiple values simultaneously
Performance	Can significantly accelerate suitable numerical workloads
Portable	Java code is not tied directly to one CPU instruction set
Explicit vectorization	Developer can express vector operations directly
JVM optimization	JIT can map operations to hardware instructions
Multiple data types	Supports byte, short, int, long, float, and double vector operations
Masks	Enables conditional lane operations
Mathematical operations	Provides many arithmetic and logical operations
Incubator API	Allows the API to evolve before finalization

23. Important Limitation

The Vector API is **not automatically faster for every program**.

It is most useful when:

Same operation
+
Many independent values
+
Suitable CPU hardware
+
Computational workload

For example:

Excellent candidate:

large arrays
↓
same mathematical operation
↓
independent elements
↓
vectorization

It may provide little benefit for code dominated by:

- I/O
- network operations
- database access
- complex object manipulation
- highly sequential dependencies

24. Java 23 Feature Summary

Property	Value
Java Version	23
Year	2024
Feature	Vector API (Eighth Incubator)
JEP	JEP 469
Status	Eighth Incubator
UniqueClassName	VectorAPI
Package	jdk.incubator.vector
Primary purpose	SIMD/vector computation
Main types	IntVector, LongVector, FloatVector, DoubleVector
Requires preview flag?	✗ No
Requires incubator module?	✓ Yes

25. In One Sentence

Java 23's Vector API (Eighth Incubator) provides a platform-independent way to express SIMD computations in Java, allowing the JVM to take advantage of modern CPU vector hardware for suitable data-parallel workloads.

The core execution model is:

```
Java Vector Code
    ↓
VectorSpecies
    ↓
Load multiple values
    ↓
Vector operation
    ↓
JIT compilation
    ↓
CPU SIMD instructions
    ↓
Multiple results simultaneously
```

Java 23 — Stream Gatherers (Second Preview)

Java Version: 23

Year: 2024

Feature: Stream Gatherers (Second Preview)

JEP: 473

Status: Second Preview

UniqueClassName: StreamGatherers

1. Definition

Stream Gatherers provide a way to create **custom intermediate operations for Java Streams**.

They allow developers to define stream-processing operations that are more sophisticated than the standard operations such as:

```
map()
filter()
flatMap()
distinct()
sorted()
limit()
```

A gatherer can **inspect, transform, combine, or aggregate stream elements while the stream is being processed.**

The main API is:

`java.util.stream.Gatherer`

and it is used through:

`Stream.gather(...)`

2. Why Were Stream Gatherers Introduced?

Java Streams provide many built-in operations, but sometimes an application needs a custom intermediate operation.

For example, suppose you want to:

- group consecutive elements
- create sliding windows
- combine elements in pairs
- maintain state while processing elements
- perform custom incremental calculations
- transform one stream element into a sequence of results

Before Gatherers, developers often had to use:

custom Collector

OR

manual loops

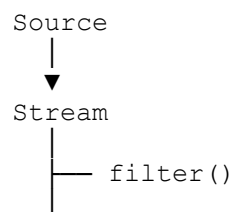
OR

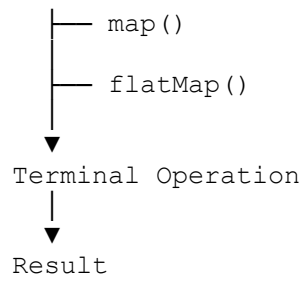
complex combinations of existing Stream operations

Gatherers provide a dedicated abstraction for **custom intermediate stream operations.**

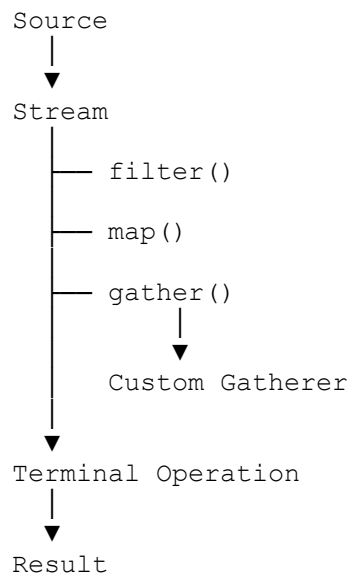
3. Stream Pipeline Before Gatherers

A traditional stream pipeline looks like:

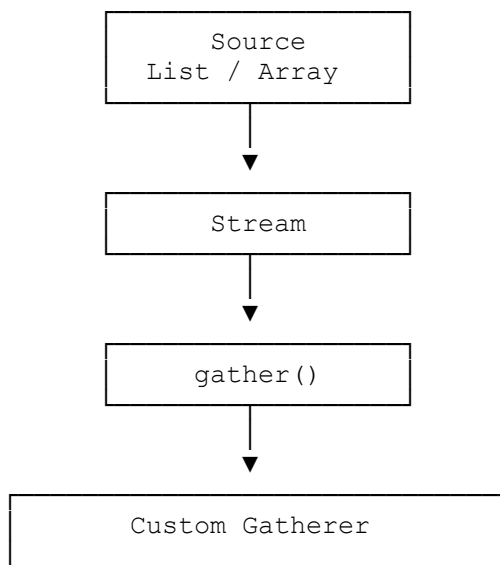


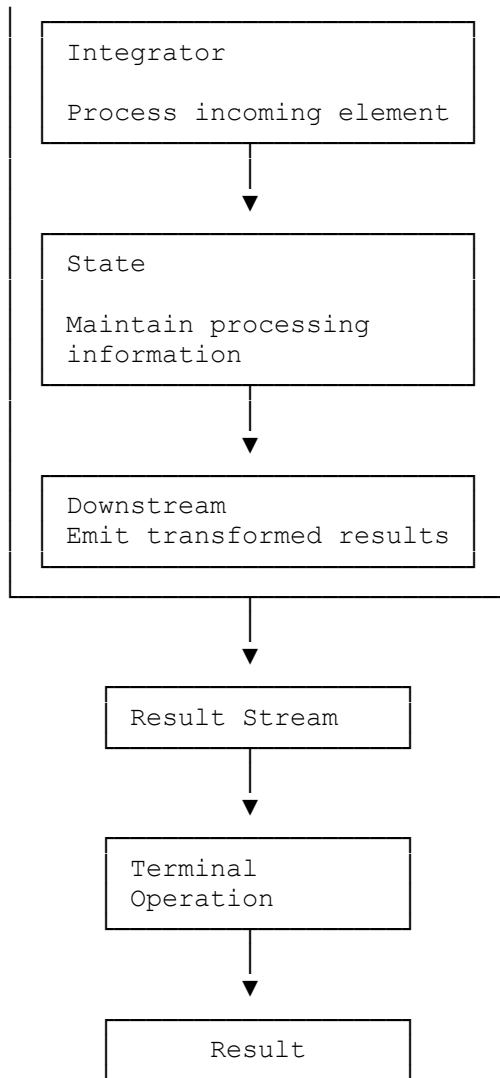


With a gatherer:



4. Flowchart





5. The Gatherer Interface

The central interface is:

Gatherer<T, A, R>

Conceptually:

Gatherer<Input, State, Output>

where:

Type	Meaning
T	Input element type

A	State/accumulator type
R	Output element type

For example:

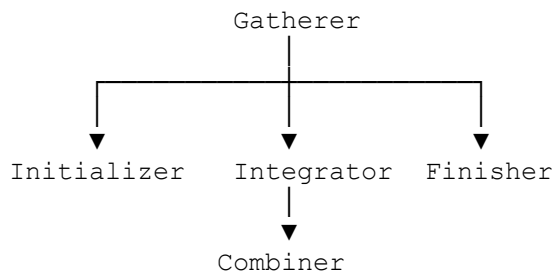
`Gatherer<Integer, List<Integer>, List<Integer>>`

could conceptually represent:

Integer input
 ↓
 List<Integer> state
 ↓
 List<Integer> output

6. Four Important Parts of a Gatherer

A gatherer can define four main behaviors:



Let's understand them.

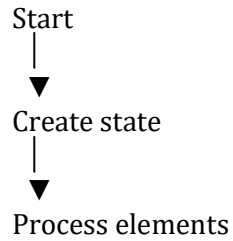
6.1 Initializer

The initializer creates the initial state.

Conceptually:

`() -> new ArrayList<>()`

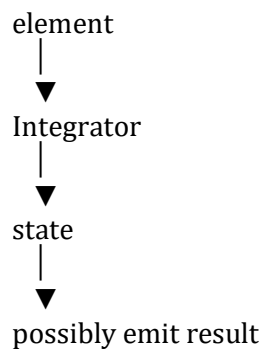
Flow:



6.2 Integrator

The **Integrator** processes each incoming element.

Conceptually:

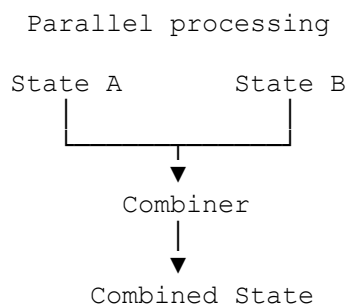


It is the primary component of a gatherer.

6.3 Combiner

The combiner is important for **parallel stream processing**.

It combines partial states:



This allows a gatherer to potentially work with parallel streams.

6.4 Finisher

The finisher is called when processing is complete.

It can emit final results based on the remaining state.

All input processed



Finisher



Final output

7. Basic Example — Windowing

One useful application of gatherers is producing **fixed-size groups/windows**.

Suppose the input is:

1 2 3 4 5 6

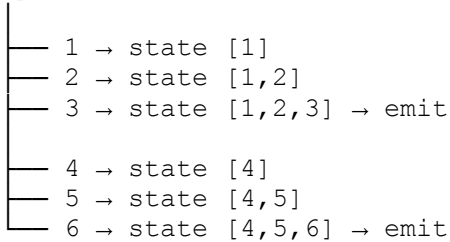
and we want groups of three:

[1, 2, 3]

[4, 5, 6]

A gatherer can maintain a temporary state:

Input



8. Java Code Example — Custom Gatherer

Java 23 provides factory methods in `Gatherer` that make simple gatherers easier to create.

Here is a custom gatherer that combines elements into pairs.

```

import java.util.List;
import java.util.stream.Gatherer;

public class StreamGatherers {

    static Gatherer<Integer, List<Integer>, List<Integer>> pairGatherer() {

        return Gatherer.ofSequential(
            () -> new java.util.ArrayList<Integer>(2),

            (state, element, downstream) -> {

                state.add(element);

                if (state.size() == 2) {

                    downstream.push(
                        List.copyOf(state)
                    );

                    state.clear();

                }

                return true;
            },

            (state, downstream) -> {

                if (!state.isEmpty()) {
                    downstream.push(
                        List.copyOf(state)
                    );
                }

            }

        );
    }

    public static void main(String[] args) {

        List<Integer> numbers =
            List.of(1, 2, 3, 4, 5, 6);

        numbers.stream()
            .gather(pairGatherer())
            .forEach(System.out::println);
    }
}

```

Output

```

[1,2]
[3,4]
[5,6]

```

9. How the Example Works

The source is:

```
List<Integer> numbers =  
    List.of(1, 2, 3, 4, 5, 6);
```

The stream is:

```
numbers.stream()
```

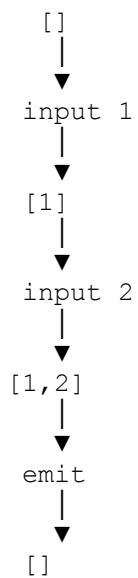
Then:

```
.gather(pairGatherer())
```

applies the custom gatherer.

The gatherer maintains state:

Initial state



Then the process repeats.

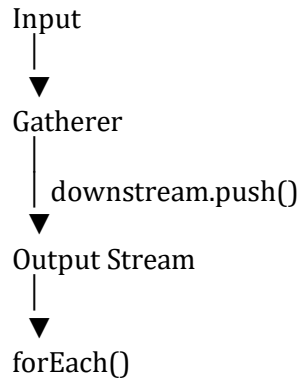
10. The `downstream.push()` Operation

One of the important concepts is:

```
downstream.push(value)
```

It sends a value from the gatherer to the next stage of the stream pipeline.

Conceptually:



For example:

```
downstream.push(List.copyOf(state));
```

means:

Send this completed list to the downstream stream operation.

11. Gatherer vs Collector

This distinction is very important.

Collector

A collector is primarily a **terminal operation**:

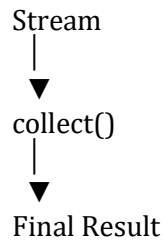
```
stream.collect(...)
```

It collects the stream into a final result.

Example:

```
List<Integer> result =  
    numbers.stream()  
        .collect(Collectors.toList());
```

Flow:

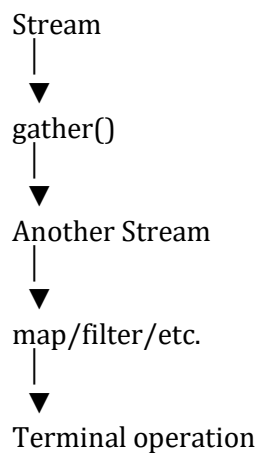


Gatherer

A gatherer is an **intermediate operation**:

`stream.gather(...)`

Flow:



Therefore:

Collector → terminal aggregation

Gatherer → intermediate transformation

12. Gatherer Pipeline

Gatherers can be combined with normal Stream operations.

```
numbers.stream()  
    .filter(n -> n > 0)  
    .gather(pairGatherer())  
    .map(Object::toString)  
    .forEach(System.out::println);
```

Conceptually:

```
List
  |
  ▼
stream()
  |
  ▼
filter()
  |
  ▼
gather()
  |
  ▼
map()
  |
  ▼
forEach()
```

This makes gatherers fit naturally into the existing Stream API.

13. Built-in Gatherers

Java's Gatherers utility class provides useful predefined gatherers.

`java.util.stream.Gatherers`

Examples include gatherers for patterns such as:

- fixed-size windows
- sliding windows
- folding
- scanning

For example, a windowing operation can be expressed using a built-in gatherer rather than implementing one yourself.

Conceptually:

```
stream.gather(
    Gatherers.windowFixed(3)
)
```

produces groups such as:

Input:

1 2 3 4 5 6

↓

[1 2 3]

[4 5 6]

14. Fixed Windows vs Sliding Windows

This is one of the useful applications.

Fixed window

Input:

1 2 3 4 5 6

Window size = 3

Result:

[1 2 3]

[4 5 6]

Sliding window

Window size = 3

Result:

[1 2 3]

[2 3 4]

[3 4 5]

[4 5 6]

Conceptually:

1 2 3 4 5 6

1 2 3

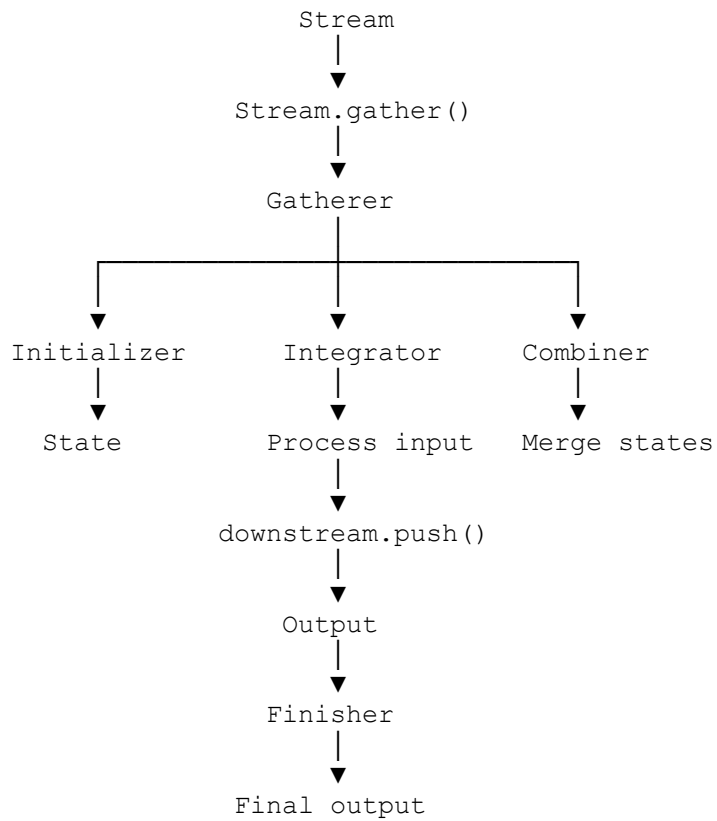
2 3 4

3 4 5

4 5 6

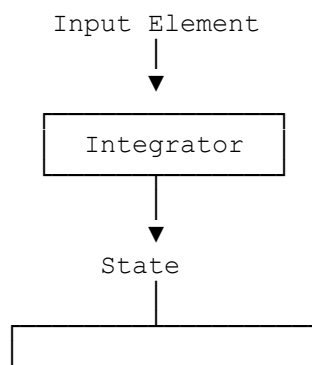
This type of processing is much more natural with gatherers.

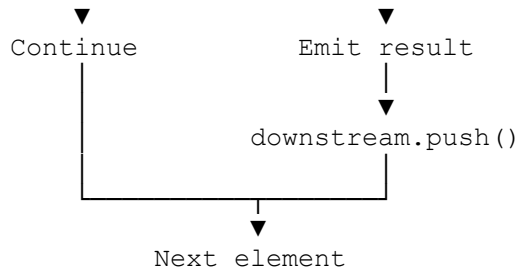
15. Gatherer Architecture



16. Gatherer Processing Model

For each incoming element:





17. State Management

A major advantage of Gatherers is that they explicitly support **stateful intermediate operations**.

For example:

```

Input:
10 20 30 40

State:
0
├── +10 → 10
├── +20 → 30
├── +30 → 60
└── +40 → 100
  
```

A gatherer can maintain that state and optionally emit intermediate results.

This enables operations such as:

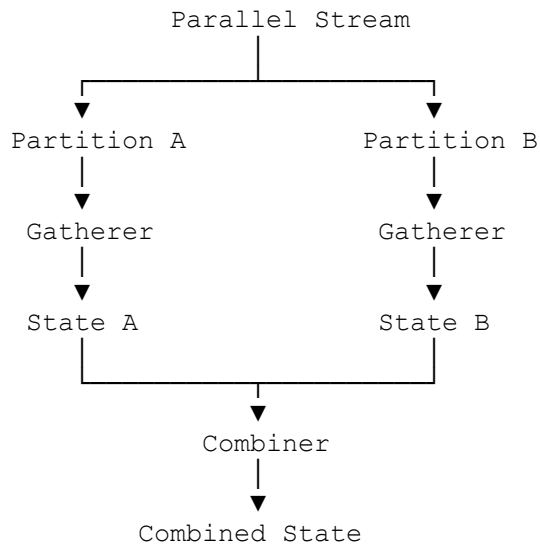
```

Running total
  ↓
Running average
  ↓
Running minimum
  ↓
Running maximum
  ↓
Custom stateful transformation
  
```

18. Gatherers and Parallel Streams

A gatherer can define a **combiner** so that partial state can be merged.

Conceptually:



However, not every stateful algorithm can be meaningfully parallelized. A gatherer's design needs to account for its combining semantics.

19. Gatherer vs map()

map() transforms each element independently:

```
numbers.stream()
  .map(n -> n * 2);
```

Flow:

```
1 → 2
2 → 4
3 → 6
4 → 8
```

A gatherer can maintain state across multiple elements:

```
1  ┌
2  └─→ [1, 2]
3  ┌
4  └─→ [3, 4]
```

Therefore:

```
map()
→ element-by-element transformation
```

gather()
→ custom stateful intermediate processing

20. Gatherer vs filter()

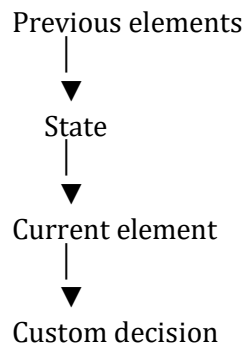
filter() decides whether an element should remain:

```
.filter(n -> n % 2 == 0)
```

Example:

1 → discard
2 → keep
3 → discard
4 → keep

A gatherer can make decisions based on **previous elements and accumulated state**.



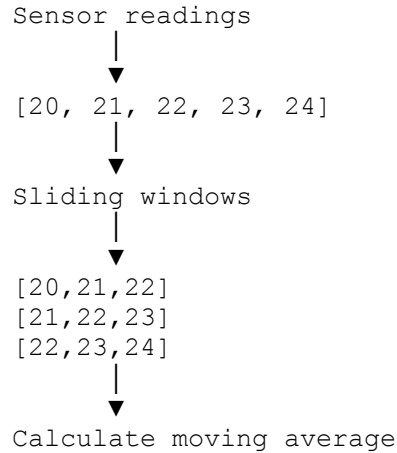
21. Real-World Applications

Stream Gatherers can be useful for:

Stream Gatherers

- Sliding windows
- Fixed-size batches
- Running calculations
- Stateful transformations
- Grouping consecutive elements
- Custom aggregation
- Incremental processing
- Event stream processing
- Custom stream algorithms

For example, processing sensor readings:



22. Complete Example — Sliding Window

A conceptual Java 23 example using the built-in gatherer:

```
import java.util.List;
import java.util.stream.Gatherers;

public class StreamGatherers {

    public static void main(String[] args) {

        List<Integer> numbers =
            List.of(1, 2, 3, 4, 5, 6);

        numbers.stream()
            .gather(Gatherers.windowSliding(3))
            .forEach(System.out::println);
    }
}
```

Output:

```
[1, 2, 3]
[2, 3, 4]
[3, 4, 5]
[4, 5, 6]
```

This demonstrates why Gatherers are useful: an operation involving **multiple neighboring stream elements** can be expressed directly within the Stream pipeline.

23. Java 23 Compilation

Because Stream Gatherers was a **second preview feature in Java 23**, compile with:

```
javac --enable-preview --release 23 StreamGatherers.java
```

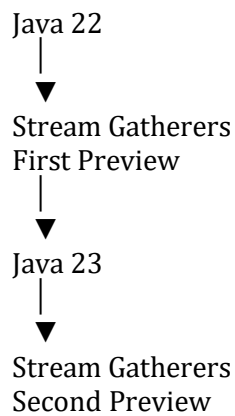
Run with:

```
java --enable-preview StreamGatherers
```

The preview flag is required because this feature had not yet been finalized in Java 23.

24. Feature Evolution

The feature evolved through preview releases:



The Java 23 version refined the API based on feedback from its first preview.

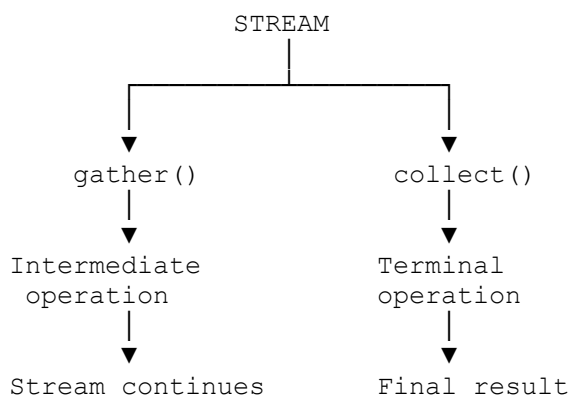
25. Key Benefits

Benefit	Description
Custom intermediate operations	Create stream operations not provided by standard Stream API
Stateful processing	Maintain state across elements
Composability	Works directly inside Stream pipelines
Downstream processing	Can emit multiple transformed results

Windowing	Supports fixed and sliding windows
Folding/scanning	Useful for incremental calculations
Parallel support	Gatherers can define state-combining behavior
Cleaner code	Avoids some manual loops for complex stream transformations
Extensibility	Developers can build reusable stream operations

26. `Stream.gather()` vs `Stream.collect()`

The difference can be remembered as:



Example:

```
stream
    .gather(customGatherer())
    .map(...)
    .filter(...)
    .collect(...);
```

Here:

`gather()` → transforms the stream
`map()` → transforms the stream
`filter()` → transforms the stream
`collect()` → finishes the stream

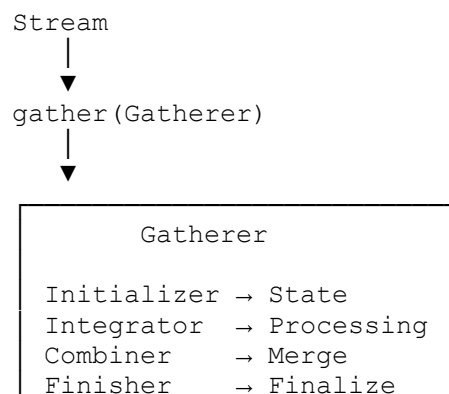
27. Java 23 Feature Summary

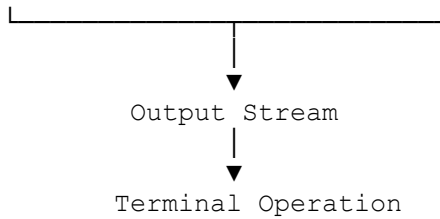
Property	Value
Java Version	23
Year	2024
Feature	Stream Gatherers
JEP	JEP 473
Status	Second Preview
UniqueClassName	StreamGatherers
Main interface	Gatherer
Main stream method	Stream.gather()
Utility class	Gatherers
Package	java.util.stream
Preview flag required?	✓ Yes

28. In One Sentence

Stream Gatherers in Java 23 provide a flexible mechanism for creating custom, potentially stateful intermediate operations in Stream pipelines, making tasks such as windowing, scanning, folding, and custom multi-element transformations much easier to express.

The core concept is:





Java 23 — Deprecate Unsafe Memory-Access Methods

Java Version: 23

Year: 2024

Feature: Deprecate Unsafe Memory-Access Methods

JEP: 471

Status: Final

UniqueClassName: UnsafeMemoryAccessDeprecation

1. Definition

Deprecate Unsafe Memory-Access Methods is a Java 23 feature that **deprecates for removal** the memory-access methods in `sun.misc.Unsafe` that were historically used for low-level manipulation of memory outside the normal Java language and JVM safety model.

The purpose is to encourage applications and libraries to migrate to safer, supported APIs, particularly the **Foreign Function & Memory (FFM) API**.

Examples of affected Unsafe capabilities include methods used for:

- Direct/off-heap memory allocation
- Reallocation
- Memory freeing
- Reading primitive values from arbitrary memory addresses
- Writing primitive values to arbitrary memory addresses

The broader direction is:

Old approach

`sun.misc.Unsafe`



Low-level memory manipulation

New approach

Foreign Function & Memory API



Supported, structured memory access

2. Why Was Unsafe Deprecated?

`sun.misc.Unsafe` provides extremely powerful low-level operations.

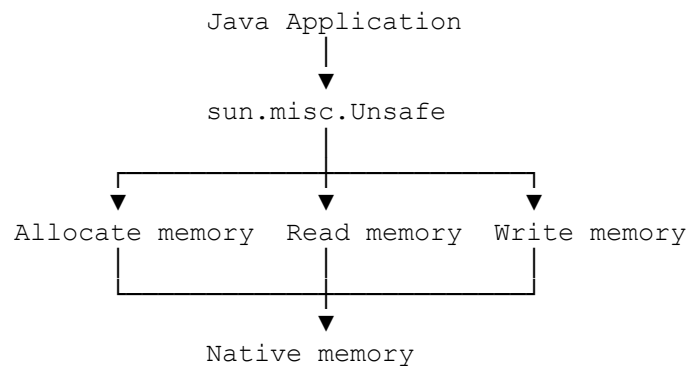
For example, code could directly allocate native memory:

```
long address = Unsafe.allocateMemory(size);
```

and later access or free it.

This bypasses many of Java's normal safety mechanisms.

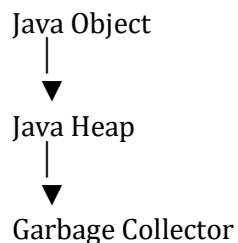
Conceptually:



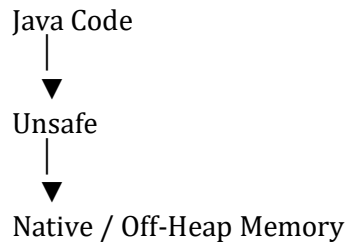
While useful for high-performance libraries and JVM infrastructure, this approach is difficult to use safely and is not a preferred long-term API.

3. Main Problem with Unsafe Memory Access

Normal Java code is designed around managed memory:



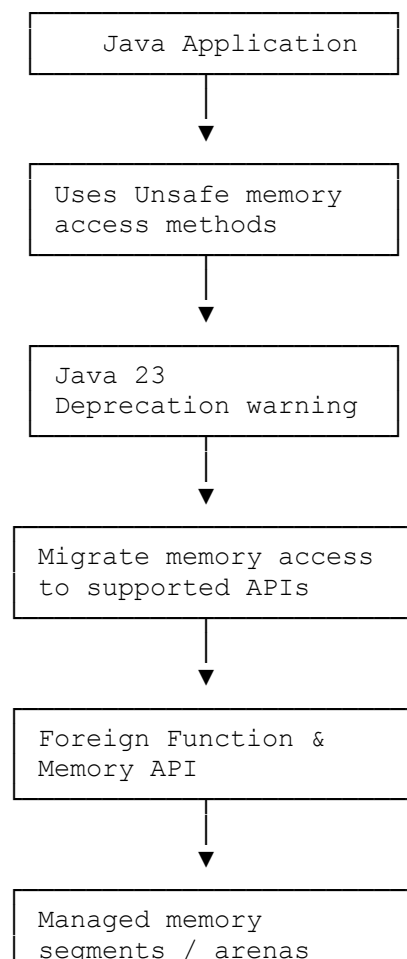
Unsafe can allow code to work outside that model:

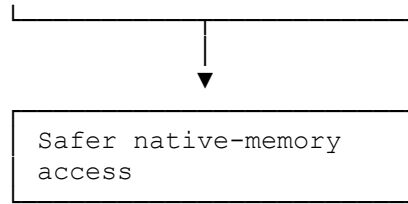


This introduces risks such as:

- Memory leaks
- Invalid memory access
- Use-after-free errors
- Incorrect memory addresses
- Difficult resource management
- Platform-dependent behavior

4. Flowchart — Java 23 Migration Direction





5. What Does "Deprecated for Removal" Mean?

There is an important distinction between:

Deprecated

and:

Deprecated for removal

Normal deprecation

Means:

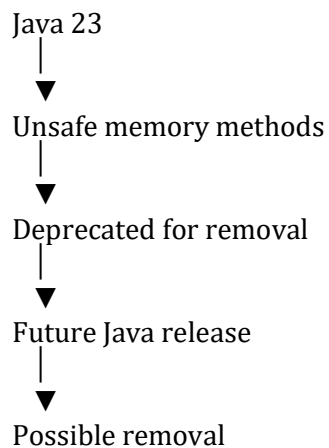
This API is discouraged, but it may remain available indefinitely.

Deprecated for removal

Means:

This API is intended to be removed in a future Java release.

Therefore:



It does **not** mean that the methods were immediately removed in Java 23.

6. Example of the Old Approach

A simplified example using Unsafe might look like:

```
import sun.misc.Unsafe;

import java.lang.reflect.Field;

public class UnsafeMemoryAccessDeprecation {

    public static void main(String[] args)
        throws Exception {

        Field field =
            Unsafe.class.getDeclaredField("theUnsafe");

        field.setAccessible(true);

        Unsafe unsafe =
            (Unsafe) field.get(null);

        long size = 8;

        long address =
            unsafe.allocateMemory(size);

        try {

            unsafe.putLong(address, 123456789L);

            long value =
                unsafe.getLong(address);

            System.out.println(
                "Value = " + value
            );

        } finally {

            unsafe.freeMemory(address);

        }

    }

}
```

Conceptual output

Value = 123456789

The important operations are:

```
unsafe.allocateMemory(...)
unsafe.putLong(...)
unsafe.getLong(...)
unsafe.freeMemory(...)
```

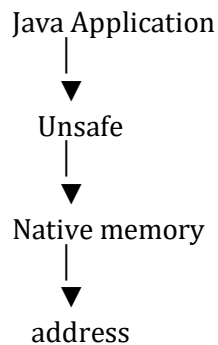
These represent the kind of low-level memory access that Java 23 is steering developers away from.

7. How the Unsafe Example Works

Step 1 — Allocate native memory

```
long address = unsafe.allocateMemory(8);
```

Conceptually:

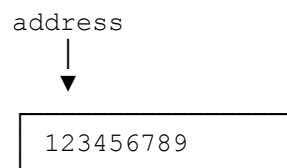


The returned value is a native memory address.

Step 2 — Write data

```
unsafe.putLong(address, 123456789L);
```

Conceptually:



Step 3 — Read data

```
long value = unsafe.getLong(address);
```

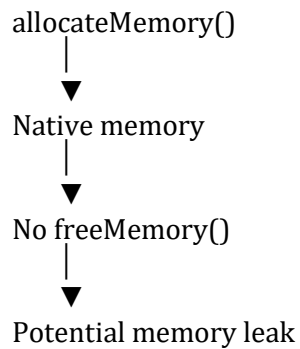
The program reads the value directly from that memory address.

Step 4 — Free memory

```
unsafe.freeMemory(address);
```

The programmer is responsible for releasing the native memory.

If this step is forgotten:



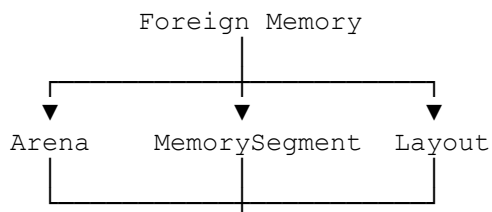
8. Recommended Modern Approach

The preferred direction is to use the **Foreign Function & Memory API**, which became final in Java 22.

The relevant APIs include:

```
java.lang.foreign.Arena  
java.lang.foreign.MemorySegment  
java.lang.foreign.ValueLayout
```

Conceptually:



▼
Structured access

9. Java Code Example — Modern Replacement

Here is a simple example using the FFM API to allocate native memory.

```
import java.lang.foreign.Arena;
import java.lang.foreign.MemorySegment;
import java.lang.foreign.ValueLayout;

public class UnsafeMemoryAccessDeprecation {

    public static void main(String[] args) {

        try (Arena arena = Arena.ofConfined()) {

            MemorySegment memory =
                arena.allocate(ValueLayout.JAVA_LONG);

            memory.set(
                ValueLayout.JAVA_LONG,
                0,
                123456789L
            );

            long value =
                memory.get(
                    ValueLayout.JAVA_LONG,
                    0
                );

            System.out.println(
                "Value = " + value
            );
        }
    }
}
```

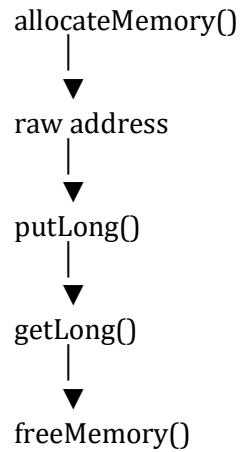
Output

Value = 123456789

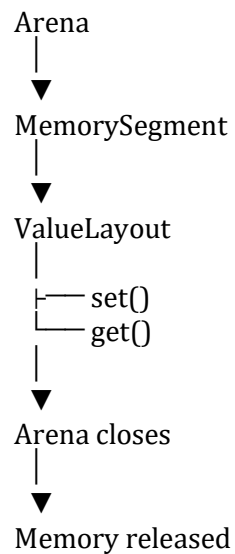
10. Why This Is Better

Compare the two approaches.

Unsafe



FFM API



The FFM API provides a more structured way to represent and manage native memory.

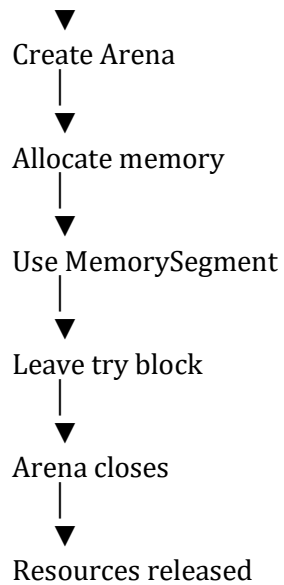
11. Arena-Based Resource Management

One major advantage is the use of an Arena.

```
try (Arena arena = Arena.ofConfined()) {  
    // allocate and use native memory  
}
```

Conceptually:

Enter try block
|



This reduces the need for explicit calls equivalent to:

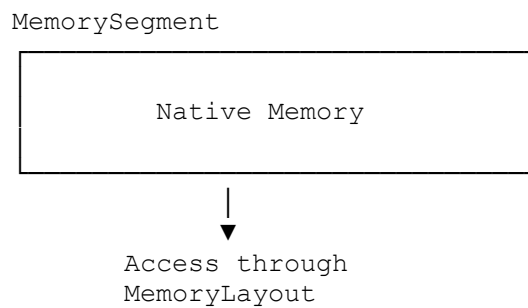
```
freeMemory()
```

12. MemorySegment

A MemorySegment provides a structured representation of a contiguous region of memory.

```
MemorySegment memory =  
    arena.allocate(ValueLayout.JAVA_LONG);
```

Conceptually:



Instead of passing around an arbitrary raw memory address, the program works with a memory segment.

13. ValueLayout

ValueLayout describes how a value is represented in memory.

For example:

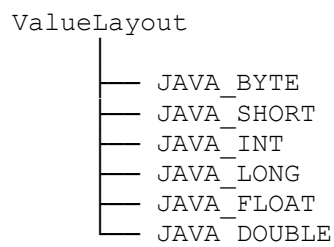
ValueLayout.JAVA_LONG

represents a Java long value.

Other layouts include concepts for:

JAVA_BYTE
JAVA_SHORT
JAVA_INT
JAVA_LONG
JAVA_FLOAT
JAVA_DOUBLE

Conceptually:



14. Comparison

Aspect	Unsafe memory methods	FFM API
API style	Low-level	Structured
Raw addresses	Common	Abstracted through segments
Resource management	Manual	Arena-based
Safety	Low-level/unsafe	More controlled
Standard supported API	✗ Unsafe is internal API	✓ Standard Java API
Native memory	✓	✓
Memory layouts	Limited/manual	MemoryLayout / ValueLayout

Future direction	Being phased out	Recommended direction

15. What Exactly Is Being Deprecated?

The JEP targets the **memory-access methods of `sun.misc.Unsafe`**, rather than declaring the entire `Unsafe` class deprecated.

The affected group includes methods associated with:

Memory allocation

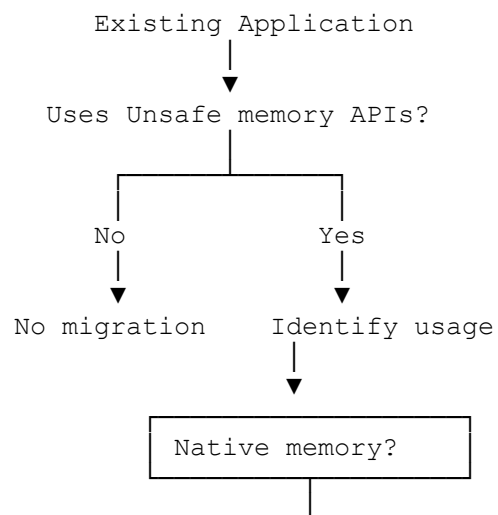
- `allocateMemory()`
- `reallocateMemory()`
- `freeMemory()`

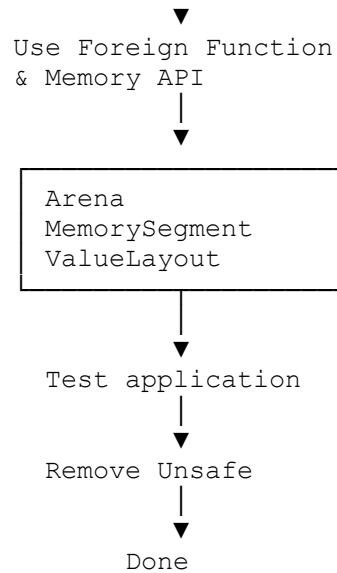
Memory access

- `getByte()`
- `getShort()`
- `getInt()`
- `getLong()`
- `getFloat()`
- `getDouble()`
- corresponding put methods

The exact set of affected methods is defined by JEP 471 and the Java 23 API documentation.

16. Migration Flowchart

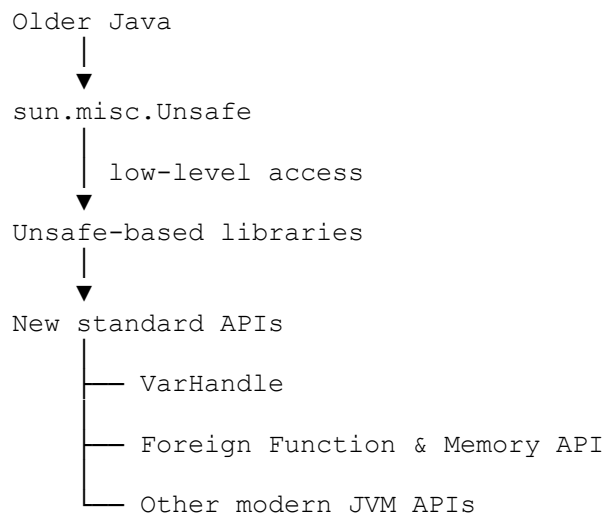




17. Why Java Is Moving Away from Unsafe

The Java platform is progressively replacing many Unsafe use cases with supported APIs.

The evolution can be viewed as:



This provides a more maintainable foundation for libraries and applications.

18. Important Clarification

Deprecation does not mean Unsafe suddenly stops working in Java 23.

Instead:

Java 23



Methods still available



Compiler/Javadoc warnings



Deprecated for removal



Developers should migrate

Existing libraries may therefore continue to work while their maintainers transition to newer APIs.

19. Real-World Impact

This feature is particularly relevant to developers maintaining:

- Serialization frameworks
- Networking libraries
- High-performance data structures
- Database drivers
- Native-memory libraries
- JVM tooling
- Bytecode/runtime frameworks
- Performance-sensitive infrastructure

A typical migration might be:

Old library



sun.misc.Unsafe



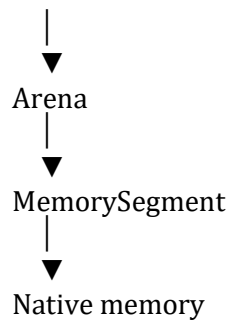
Native memory

becoming:

New library



Foreign Function & Memory API



20. Java 23 Compilation

The modern FFM example uses a standard Java API that was finalized in Java 22, so it does **not** require an incubator or preview flag in Java 23.

Compile normally:

```
javac UnsafeMemoryAccessDeprecation.java
```

Run:

```
java UnsafeMemoryAccessDeprecation
```

21. Key Benefits

Benefit	Explanation
Safer direction	Moves developers away from arbitrary memory manipulation
Standard API	Encourages supported Java APIs
Better resource management	Arena provides structured lifetime management
Memory abstraction	MemorySegment represents memory regions
Explicit layouts	ValueLayout describes data representation
Easier maintenance	Less dependence on internal APIs
Future compatibility	Reduces dependence on APIs intended for removal

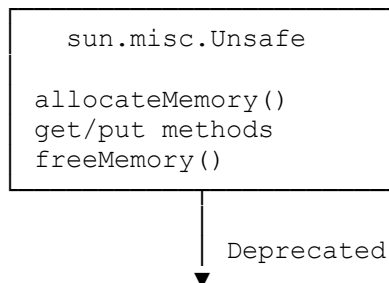
22. Feature Summary

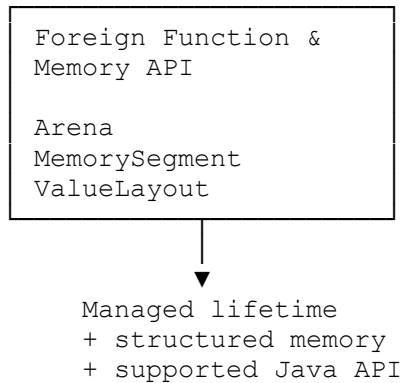
Property	Value
Java Version	23
Year	2024
Feature	Deprecate Unsafe Memory-Access Methods
JEP	JEP 471
Status	Final
UniqueClassName	UnsafeMemoryAccessDeprecation
Affected API	Selected sun.misc.Unsafe memory-access methods
Primary replacement	Foreign Function & Memory API
Replacement package	java.lang.foreign
Deprecated for removal?	✓ Yes
Immediately removed?	✗ No
Preview feature?	✗ No

23. In One Sentence

Java 23's JEP 471 deprecates for removal the memory-access methods of `sun.misc.Unsafe`, encouraging developers to migrate low-level native-memory operations to the safer and standardized Foreign Function & Memory API.

The overall migration direction is:





Java 23 — ZGC Generational Mode by Default

Java Version: 23

Year: 2024

Feature: ZGC Generational Mode by Default

JEP: 474

Status: Final

UniqueClassName: GenerationalZGC

1. Definition

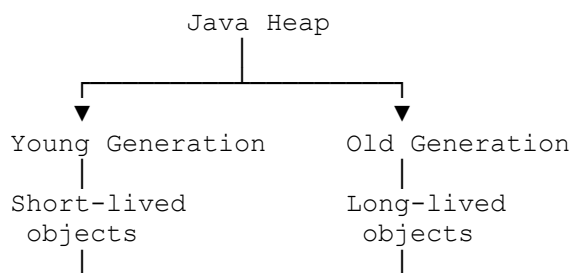
Generational ZGC is the generational mode of the **Z Garbage Collector (ZGC)**, where the heap is divided conceptually into **young** and **old generations**.

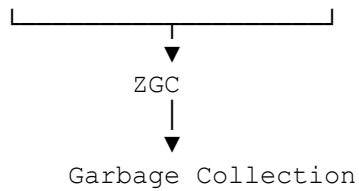
In **Java 23**, generational mode became the **default mode of ZGC**.

The main idea is based on the **generational hypothesis**:

Most newly created objects become unreachable relatively quickly, while objects that survive longer are more likely to remain alive.

Therefore, instead of treating all objects equally, Generational ZGC can focus collection work on younger objects more frequently.





2. What Is ZGC?

ZGC (Z Garbage Collector) is a low-latency garbage collector designed for applications that need very small GC pauses, including applications with very large heaps.

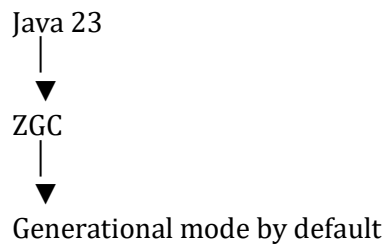
Its goals include:

- Low pause times
- Concurrent garbage collection
- Large heap support
- Good scalability
- Reduced application interruption

Before Java 23, ZGC supported both:

Non-generational ZGC
Generational ZGC

Java 23 changes the default:



3. What Changed in Java 23?

Before Java 23, generational ZGC had to be explicitly enabled.

Conceptually:

`java -XX:+UseZGC -XX:+ZGenerational MyApplication`

In Java 23:

```
java -XX:+UseZGC MyApplication
```

uses **generational ZGC by default**.

The old non-generational mode can be selected explicitly:

```
java -XX:+UseZGC -XX:-ZGenerational MyApplication
```

So the default changed from:

ZGC
└─ Non-generational by default

to:

ZGC
└─ Generational by default

4. Why Generational ZGC?

Consider an application creating millions of temporary objects.

```
for (...) {  
    String temp = ...;  
    process(temp);  
}
```

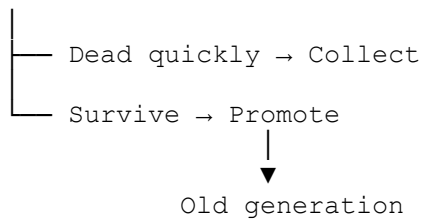
Many of these objects may become unreachable very quickly.

Without generational collection:

All objects
┆
▼
ZGC considers heap broadly
┆
▼
Identify garbage

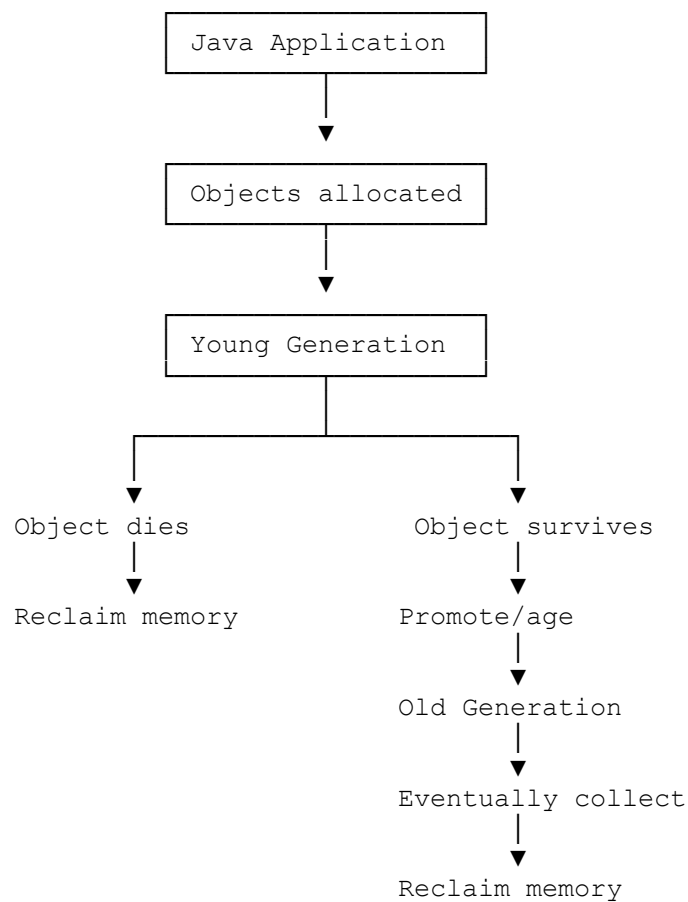
With generational ZGC:

New objects
┆
▼
Young generation



This can make garbage collection more efficient.

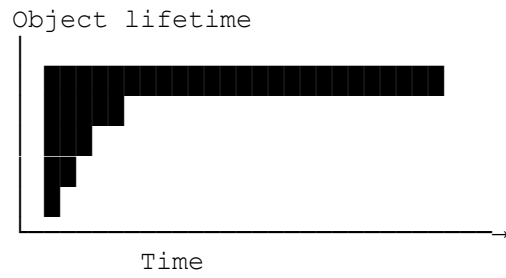
5. Flowchart



6. Generational Hypothesis

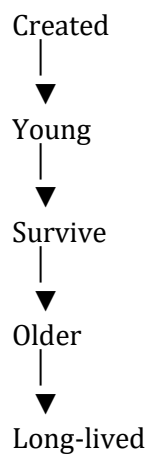
Generational GC is based on two observations.

Most objects die young



Many objects have a short lifetime.

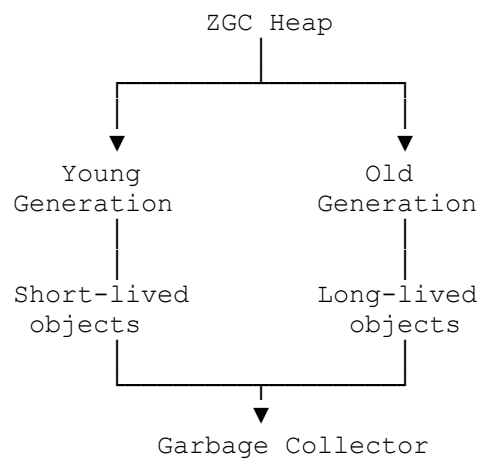
Some objects live for a long time



Generational collectors exploit this difference.

7. Young and Old Generations

Conceptually:



An important point is that ZGC remains a **low-latency concurrent collector**; generational mode adds generational organization and collection strategies rather than turning ZGC into a traditional stop-the-world generational collector.

8. Java Code Example

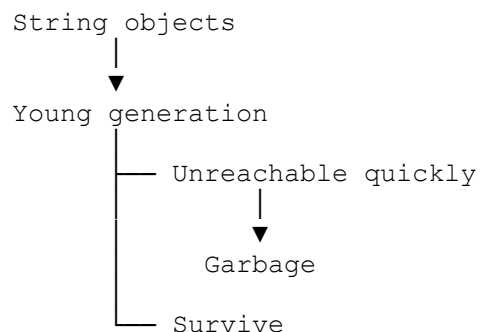
The Java source code itself does **not** normally change when using Generational ZGC.

For example:

```
public class GenerationalZGC {  
    public static void main(String[] args) {  
        System.out.println("Generational ZGC demonstration");  
        for (int i = 0; i < 10_000_000; i++) {  
            // Create short-lived objects  
            String value =  
                "Object-" + i;  
            if (i % 1_000_000 == 0) {  
                System.out.println(  
                    "Created: " + i + " objects"  
                );  
            }  
        }  
        System.out.println("Application finished.");  
    }  
}
```

This program continuously creates temporary String objects.

Conceptually:



↓
Older generation

9. Running with ZGC

Compile:

```
javac GenerationalZGC.java
```

Run using ZGC:

```
java -XX:+UseZGC GenerationalZGC
```

In **Java 23**, generational mode is enabled by default when ZGC is selected.

You can explicitly specify it as well:

```
java -XX:+UseZGC -XX:+ZGenerational GenerationalZGC
```

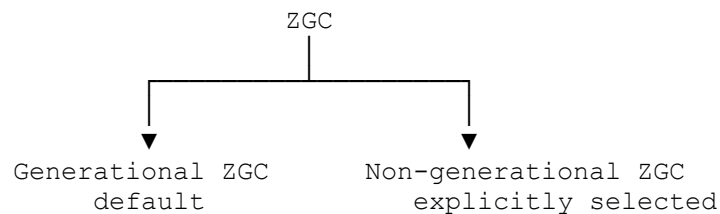
The explicit option is useful when you want to make the configuration obvious, although it is unnecessary in Java 23's default ZGC configuration.

10. Disabling Generational Mode

Java 23 also provides a way to explicitly select the non-generational mode:

```
java -XX:+UseZGC -XX:-ZGenerational GenerationalZGC
```

Therefore:



11. Creating Long-Lived Objects

We can create objects that remain reachable for a longer period.

```

import java.util.ArrayList;
import java.util.List;

public class GenerationalZGC {

    public static void main(String[] args) {

        List<byte[]> longLivedObjects =
            new ArrayList<>();

        for (int i = 0; i < 100; i++) {

            // Long-lived objects
            longLivedObjects.add(
                new byte[1024 * 1024]
            );

            // Temporary object
            String temporary =
                "Temporary-" + i;

            System.out.println(
                "Iteration: " + i
            );

        }

        System.out.println(
            "Long-lived objects: "
            + longLivedObjects.size()
        );

    }
}

```

Here:

temporary
 ↓
 ▼
 Short lifetime
 ↓
 ▼
 Potential young-generation garbage

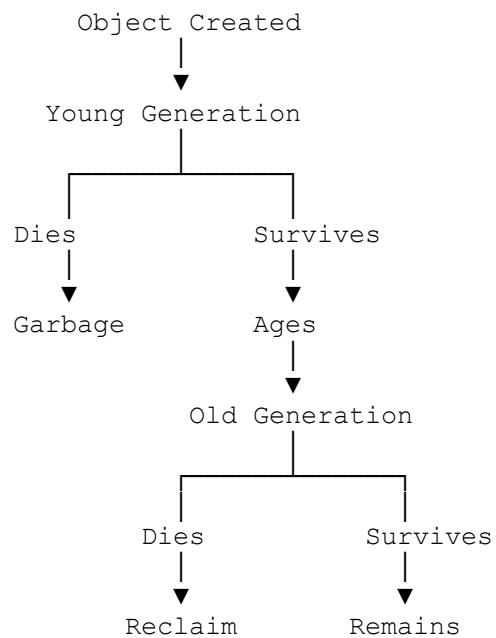
while:

longLivedObjects
 ↓
 ▼
 References remain
 ↓
 ▼
 Objects stay alive

↓
Potentially become old

12. Object Lifecycle

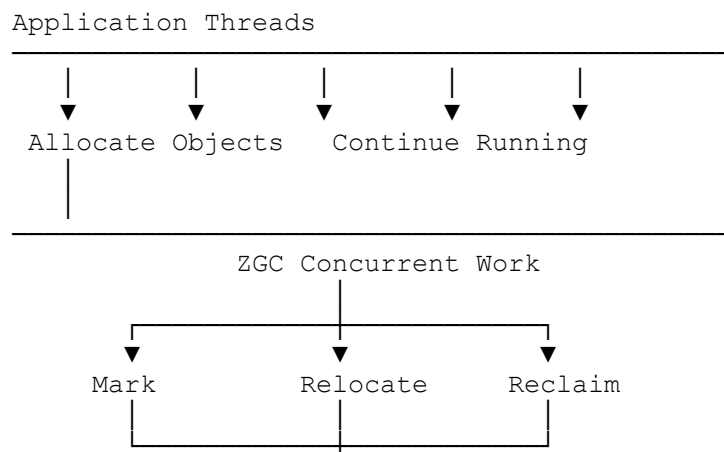
A simplified conceptual lifecycle is:



13. ZGC Collection Process

Generational ZGC continues to emphasize **concurrent collection**.

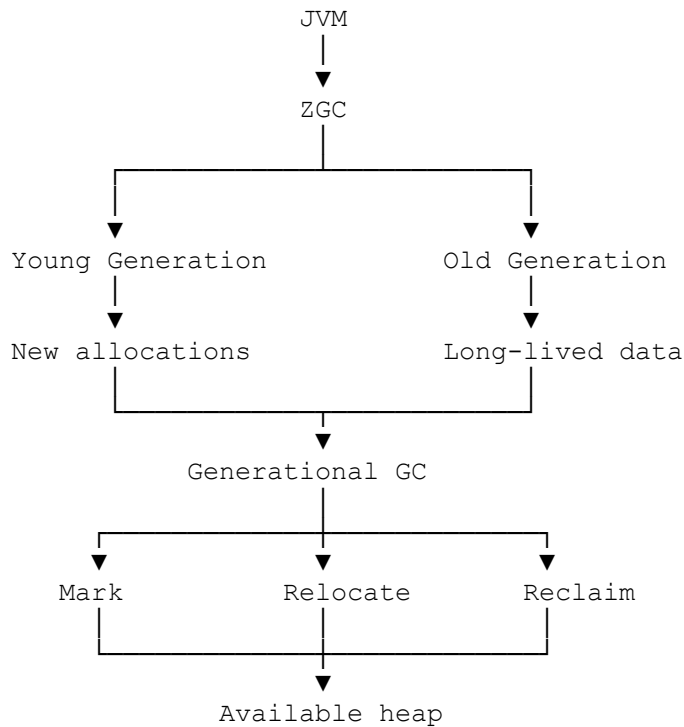
A simplified view:



▼
Continue Application

The exact internal implementation is considerably more sophisticated, but the important concept is that ZGC performs much of its GC work concurrently with application execution.

14. Generational ZGC Architecture



15. Why This Can Improve Performance

Suppose an application has:

1,000,000 objects

and:

900,000 objects

are short-lived.

A generational collector can focus more frequently on the region containing newer objects.

Conceptually:

Without generational approach

```
1,000,000 objects
├─ 900,000 short-lived
└─ 100,000 long-lived
```

With generational approach

```
Young
├─ 900,000 mostly short-lived
└─ frequently collected

Old
└─ 100,000 long-lived
    └─ treated differently
```

This can reduce unnecessary work on long-lived objects.

16. Performance Characteristics

Generational ZGC is particularly useful for applications with:

- High allocation rates
- Many short-lived objects
- Large heaps
- Latency-sensitive workloads
- Server applications
- Microservices
- Large Java applications

Potential benefits include:

More efficient garbage collection



Better allocation handling



Less work on long-lived objects



Potentially better throughput



Low-latency characteristics retained

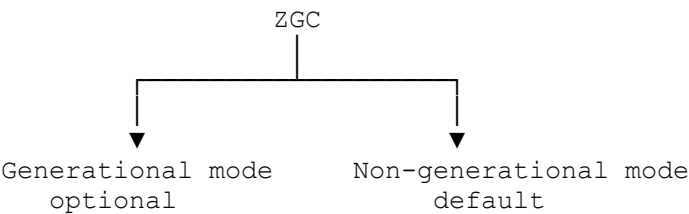
Actual performance depends heavily on the workload, heap size, allocation patterns, and hardware.

17. Generational ZGC vs Traditional Generational GC

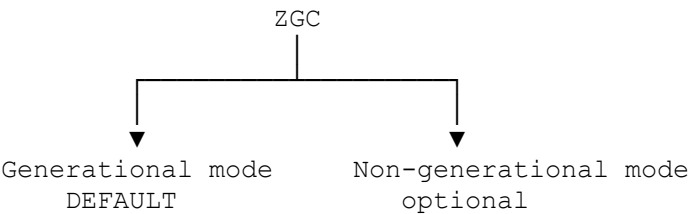
Aspect	Traditional Generational GC	Generational ZGC
Young generation	✓	✓
Old generation	✓	✓
Generational hypothesis	✓	✓
Concurrent collection	Depends on collector	Core ZGC characteristic
Low-latency focus	Collector-dependent	Strong focus
Large heaps	Varies	Strong focus
Java 23 ZGC default	—	✓ Generational

18. ZGC Before and After Java 23

Before Java 23



Java 23



This is the key change introduced by JEP 474.

19. JVM Configuration

Use ZGC

```
java -XX:+UseZGC GenerationalZGC
```

Explicitly enable generational ZGC

```
java -XX:+UseZGC -XX:+ZGenerational GenerationalZGC
```

Explicitly disable generational mode

```
java -XX:+UseZGC -XX:-ZGenerational GenerationalZGC
```

For Java 23, the first command is normally sufficient for generational ZGC.

20. Checking the Garbage Collector

You can run the application with GC logging:

```
java -Xlog:gc -XX:+UseZGC GenerationalZGC
```

For more detailed information:

```
java -Xlog:gc*=debug -XX:+UseZGC GenerationalZGC
```

This allows you to observe garbage-collection activity.

21. Memory Flow Example

Imagine an application continuously allocating:

```
A B C D E F G H I J
```

Initially:

```
Young Generation
```

```
[A B C D E F G H I J]
```

After some objects become unreachable:

Young Generation

```
[A X C X X F X H X J]
      ↑      ↑
    garbage
```

Collection can reclaim the dead objects.

Surviving objects can age toward the old generation:

```
Young
  |
  | survive
  ▼
Old
```

Meanwhile, newly allocated objects continue entering the young generation.

22. Important Clarification

Generational ZGC does not mean that Java application developers need to manually move objects between generations.

The JVM manages this automatically.

You write:

```
new MyObject();
```

and the JVM/GC determines how that object is managed.

```
Java Code
  |
  ▼
new Object()
  |
  ▼
JVM
  |
  ▼
Generational ZGC
  ├── Young
  ├── Aging
  └── Old
```

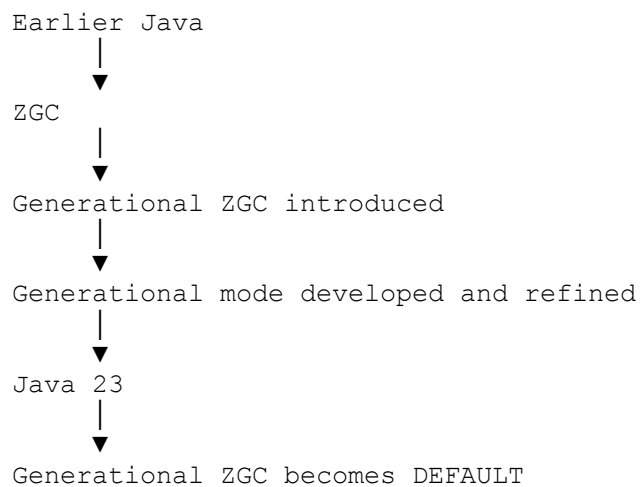
There is normally no application-level API for manually promoting an object.

23. Main Advantages

Advantage	Description
Generational collection	Separates young and old object populations
Better allocation efficiency	Targets short-lived objects effectively
Low latency	Retains ZGC's low-pause design
Large heap support	Suitable for large-memory applications
Concurrent processing	Much GC work occurs concurrently
Better throughput potential	Less unnecessary work on long-lived objects
Automatic management	Developers do not manually manage generations
Java 23 default	Generational mode is the default when using ZGC

24. Feature Evolution

The progression can be summarized as:



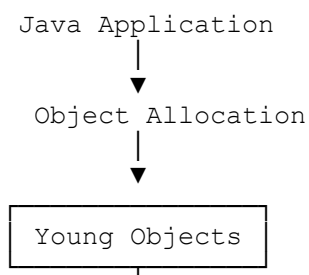
25. Java 23 Feature Summary

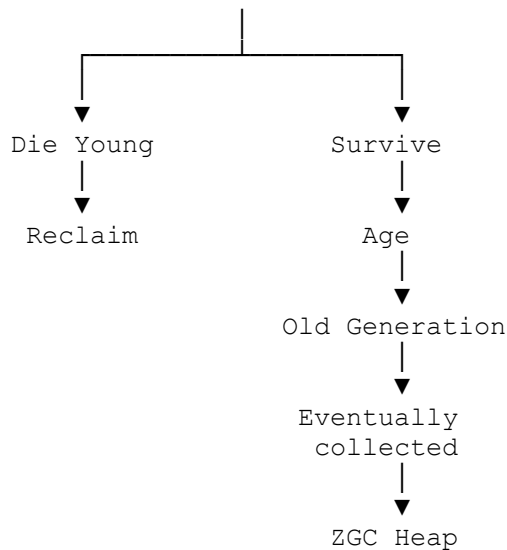
Property	Value
Java Version	23
Year	2024
Feature	ZGC Generational Mode by Default
JEP	JEP 474
Status	Final
UniqueClassName	GenerationalZGC
Garbage Collector	ZGC
Generational mode	Default
Young generation	✓
Old generation	✓
Low-latency design	✓
Concurrent GC	✓
Preview feature?	✗ No
Incubator feature?	✗ No

26. In One Sentence

Java 23 makes Generational ZGC the default mode of ZGC, allowing the collector to exploit the generational behavior of Java objects—collecting short-lived objects more effectively while continuing to provide ZGC's low-latency, largely concurrent garbage-collection design.

Core concept





Key takeaway: The Java 23 change is primarily a **change in ZGC's default operating mode**, not a new Java language syntax. Your Java source code generally stays the same; you select ZGC with `-XX:+UseZGC`, and Java 23 uses its generational mode by default.

Java 23 — Module Import Declarations (Preview)

Java Version: 23

Year: 2024

Feature: Module Import Declarations (Preview)

JEP: 476

Status: Preview

UniqueClassName: ModuleImportDeclarations

1. Definition

Module Import Declarations allow a Java source file to import **all the accessible public types exported by a module** using a single module-level import declaration.

The syntax is:

```
import module java.base;
```

Instead of individually importing classes:

```
import java.util.List;
```

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.Set;
```

you can use:

```
import module java.base;
```

and make the exported public types of that module available for simple-name use, subject to Java's normal import and name-resolution rules.

The feature is intended to make Java source code—particularly **small programs and beginner-friendly examples**—less verbose.

2. Why Module Import Declarations?

Consider a simple Java program using several classes from different packages in the same module.

Traditional imports:

```
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.HashMap;
```

Module import:

```
import module java.base;
```

Conceptually:

Traditional

```
├── import List
├── import ArrayList
├── import Map
└── import HashMap
```

Module Import

```
└── import module java.base
    │
    ▼
    Accessible exported types
```

3. Important Distinction

A **module import** is not the same thing as a **wildcard package import**.

Traditional wildcard:

```
import java.util.*;
```

means:

Import accessible types from **one package**, java.util.

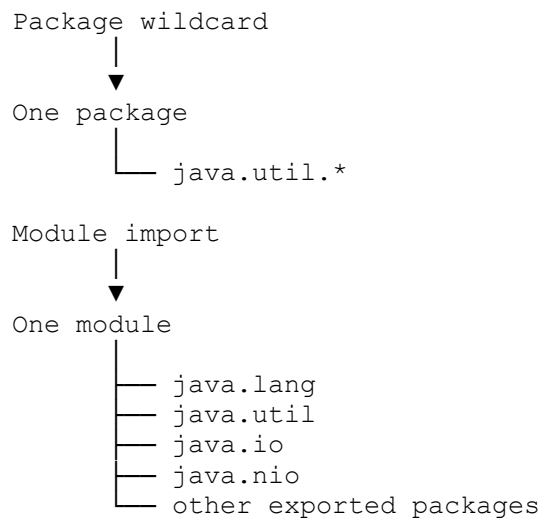
Module import:

```
import module java.base;
```

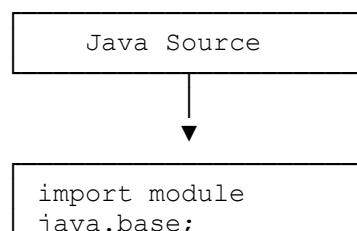
means:

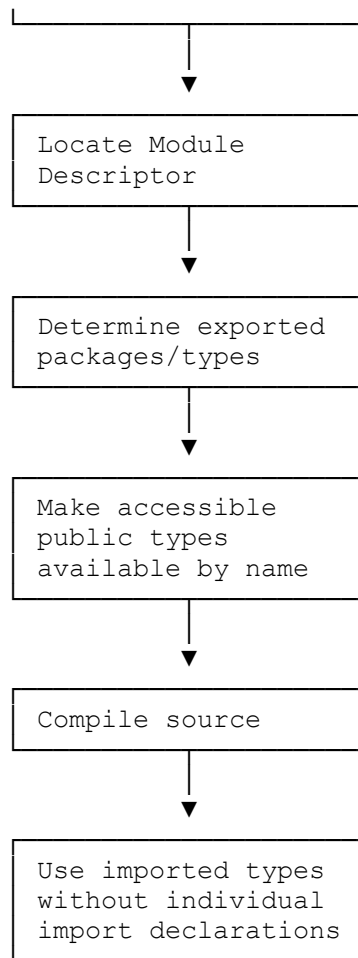
Import accessible public types exported by **the entire module**.

So:



4. Flowchart





5. Basic Syntax

The syntax is:

```
import module moduleName;
```

For example:

```
import module java.base;
```

Here:

```
import
  |
  ▼
module
  |
  ▼
```

```
java.base
|
▼
;
```

The keyword `module` tells the compiler that this is a **module import**, not a normal package or type import.

6. Java Code Example

Here is a simple Java 23 example:

```
import module java.base;

public class ModuleImportDeclarations {

    public static void main(String[] args) {

        List<String> names =
            new ArrayList<>();

        names.add("Anurag");
        names.add("Java");
        names.add("Module");

        for (String name : names) {
            System.out.println(name);
        }
    }
}
```

Notice that we did **not** write:

```
import java.util.List;
import java.util.ArrayList;
```

Instead:

```
import module java.base;
```

provides access to the relevant public types exported by `java.base`.

Output

```
Anurag
Java
Module
```

7. How It Works

The statement:

```
import module java.base;
```

tells the compiler to consider the accessible public types exported by `java.base`.

For example, `java.base` contains packages such as:

```
java.lang  
java.io  
java.math  
java.net  
java.nio  
java.time  
java.util  
...
```

Therefore, code can use types such as:

```
List  
ArrayList  
HashMap  
String  
System  
LocalDate  
BigInteger
```

without necessarily writing individual import declarations.

8. Traditional Imports vs Module Import

Traditional

```
import java.util.ArrayList;  
import java.util.HashMap;  
import java.util.List;  
import java.util.Map;
```

Package wildcard

```
import java.util.*;
```

Module import

```
import module java.base;
```

The scope increases:

Single Type



ArrayList

Package



java.util.*

Module



java.base

9. Example Using Multiple APIs

```
import module java.base;

public class ModuleImportDeclarations {

    public static void main(String[] args) {

        List<String> languages =
            new ArrayList<>();

        languages.add("Java");
        languages.add("Python");
        languages.add("Go");

        Map<String, Integer> lengths =
            new HashMap<>();

        for (String language : languages) {
            lengths.put(
                language,
                language.length()
            );
        }

        System.out.println(lengths);
    }
}
```

Possible output:

```
{Java=4, Python=6, Go=2}
```

Here the program uses:

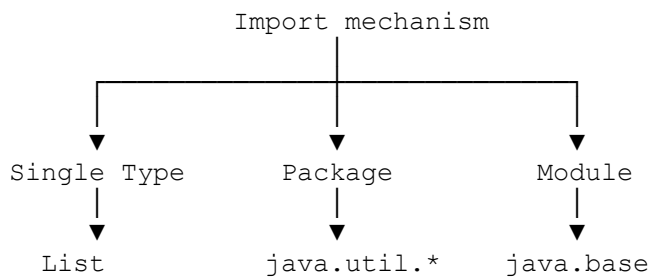
```
java.util.List  
java.util.ArrayList  
java.util.Map  
java.util.HashMap
```

through the module import.

10. Module Import vs Package Import

Import	Meaning
import java.util.List;	Import one type
import java.util.*;	Import types from java.util
import module java.base;	Import accessible exported types from java.base

Visual comparison:

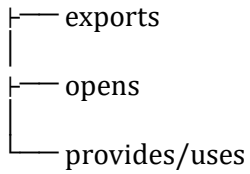


11. Why Modules Matter

Java's module system was introduced in **Java 9** through the Java Platform Module System (JPMS).

A module can declare:

```
Module  
|  
├─ requires  
|
```



For example, a module descriptor could contain:

```
module com.example.app {  
    requires java.sql;  
    exports com.example.api;  
}
```

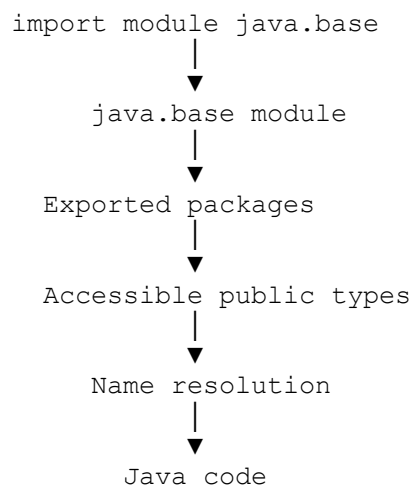
A module import operates within this module system.

12. Module Import Resolution

Suppose:

```
import module java.base;
```

The compiler conceptually performs:



It does **not** mean:

"Import every class from every package regardless of accessibility."

Java's module accessibility rules still apply.

13. Important Name-Conflict Example

Suppose two imported modules contain types with the same simple name.

Conceptually:

```
Module A
└─ example.Foo

Module B
└─ other.Foo
```

If both modules are imported:

```
import module moduleA;
import module moduleB;
```

then:

Foo value;

could become ambiguous.

The developer can resolve the ambiguity using a qualified name or an explicit import.

This is one reason module imports should be used thoughtfully.

14. Explicit Imports Take Precedence

Module imports do not eliminate the existing import mechanisms.

You can combine:

```
import module java.base;
import java.util.concurrent.ExecutorService;
```

The explicit type import can be used when you want a particular type to be unambiguous or clearly documented.

Conceptually:

```
Module import
└─ broad availability
```

Explicit import
└─ precise selection

15. Module Import with java.base

java.base is particularly useful because it is the fundamental Java platform module.

It provides core packages such as:

java.lang
java.io
java.math
java.net
java.nio
java.time
java.util

A simple educational program can therefore become:

```
import module java.base;

public class ModuleImportDeclarations {
    public static void main(String[] args) {
        List<Integer> numbers =
            List.of(10, 20, 30, 40);

        int total = 0;

        for (int number : numbers) {
            total += number;
        }

        System.out.println(
            "Total = " + total
        );
    }
}
```

Output:

Total = 100

16. Relationship with Implicit Imports

This feature is especially interesting alongside Java's evolving support for **implicitly declared classes and instance main methods**.

A beginner-friendly Java program can potentially become very small.

Conceptually:

```
Traditional Java program
├── package
├── imports
├── class declaration
└── main method
```

Modern simplified Java:

```
Module import
+
Implicit class
+
Instance main
↓
Very small Java program
```

This is one of the motivations for the feature: **reduce ceremony in introductory and small Java programs**.

17. Complete Example

```
import module java.base;

public class ModuleImportDeclarations {

    public static void main(String[] args) {

        List<String> products = List.of(
            "Laptop",
            "Mobile",
            "Tablet"
        );

        Map<String, Integer> inventory =
            new HashMap<>();

        for (String product : products) {
```

```
        inventory.put(
            product,
            product.length()
        );
    }

    System.out.println(
        "Products: " + products
    );

    System.out.println(
        "Inventory: " + inventory
    );
}
}
```

Possible output:

Products: [Laptop, Mobile, Tablet]
Inventory: {Laptop=6, Mobile=6, Tablet=6}

18. Compilation

Because **Module Import Declarations** is a **Java 23 preview feature**, enable preview support.

Compile:

```
javac --enable-preview --release 23 ModuleImportDeclarations.java
```

Run:

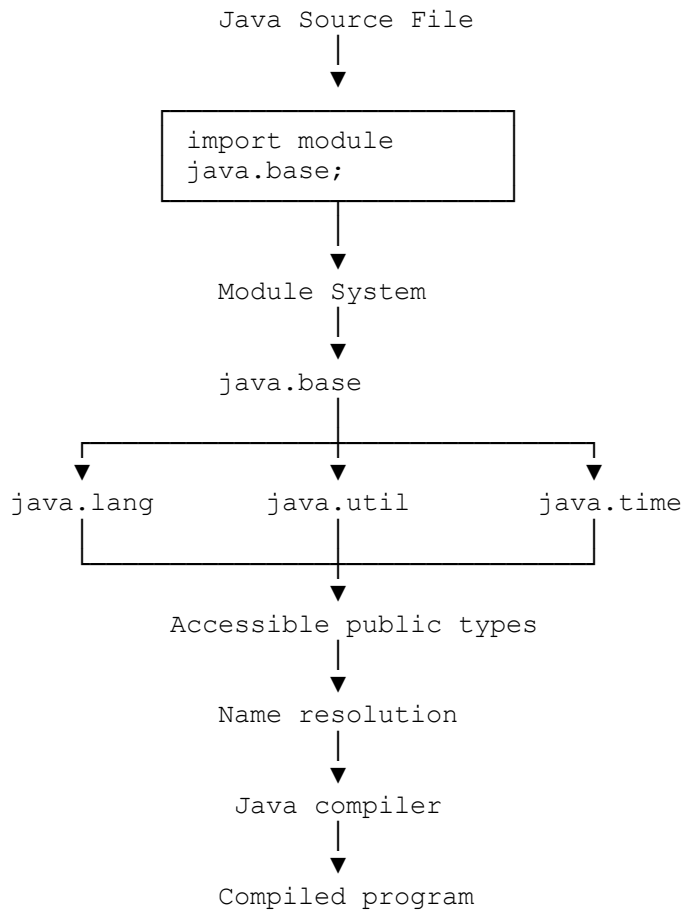
```
java --enable-preview ModuleImportDeclarations
```

The important option is:

`--enable-preview`

because the feature had not yet been finalized in Java 23.

19. Conceptual Architecture



20. Benefits

Benefit	Explanation
Less boilerplate	Fewer individual import declarations
Module-level abstraction	Imports are organized around modules
Convenient for small programs	Particularly useful for examples and learning
Works with JPMS	Integrates with Java's module system
Multiple packages	One module import can cover types across exported packages
Explicit option remains	Normal imports can still be used
Better readability	Small programs can be more concise

21. Important Limitations

A module import does **not**:

- Import inaccessible types.
- Bypass module boundaries.
- Import non-exported packages.
- Eliminate name conflicts.
- Replace every explicit import in large applications.
- Automatically make every type in the JDK available without regard to accessibility.

The module system continues to enforce accessibility.

22. Module Import vs import module

Be careful with the syntax.

Package import

```
import java.util.*;
```

Static import

```
import static java.lang.Math.*;
```

Module import

```
import module java.base;
```

The keyword:

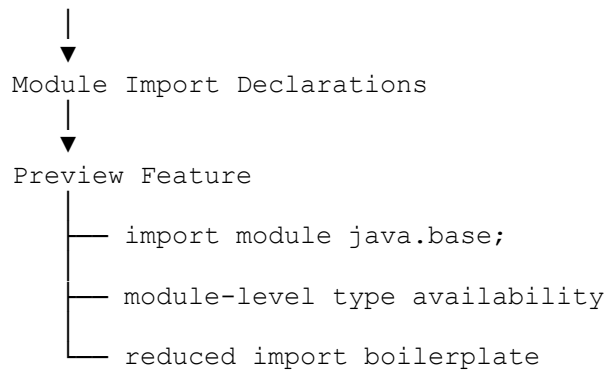
`module`

is what distinguishes the new construct.

23. Feature Evolution

The feature's development can be represented as:

```
Java 23  
  |  
  ▼  
JEP 476
```



It is part of a broader effort to make Java easier to use for **small, educational, and introductory programs**.

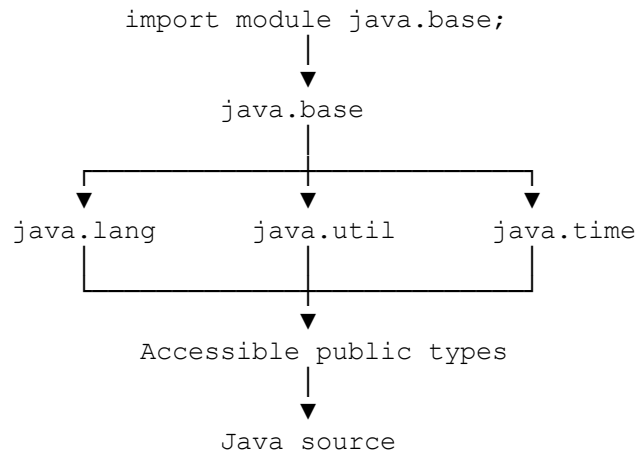
24. Java 23 Feature Summary

Property	Value
Java Version	23
Year	2024
Feature	Module Import Declarations
JEP	JEP 476
Status	Preview
UniqueClassName	ModuleImportDeclarations
Syntax	import module module.name;
Example	import module java.base;
Purpose	Import accessible exported types from a module
Related technology	Java Platform Module System (JPMS)
Preview flag required?	✓ Yes

25. In One Sentence

Module Import Declarations in Java 23 allow a source file to import the accessible public types exported by an entire module using `import module module.name;`, reducing import boilerplate while respecting Java's module accessibility rules.

Core concept



Key takeaway: `import module` operates at the **module level**, whereas `import java.util.*` operates only at the **package level**.

Java 23 — Implicitly Declared Classes and Instance Main Methods (Third Preview)

Java Version: 23

Year: 2024

Feature: Implicitly Declared Classes and Instance Main Methods (Third Preview)

JEP: 477

Status: Third Preview

UniqueClassName: `ImplicitClassesInstanceMain`

1. Definition

Implicitly Declared Classes and Instance Main Methods is a Java feature that allows developers—especially beginners—to write small Java programs **without explicitly declaring a class** and without requiring the traditional:

```
public static void main(String[] args)
```

method.

Instead, a simple Java program can look like:

```
void main() {  
    System.out.println("Hello, Java 23!");  
}
```

The Java compiler treats the source as declaring an **implicitly declared class**, and the `main()` method can be an **instance method** rather than a static method.

This significantly reduces the amount of Java boilerplate.

2. Traditional Java Program

Before this feature, even the smallest Java program normally looked like:

```
public class HelloWorld {  
  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

There is considerable ceremony:

```
public class HelloWorld  
    |  
    |— class declaration  
    |  
    |— public static void main(String[] args)  
        |  
        |— public  
        |— static  
        |— void  
        |— String[]
```

For beginners, much of this syntax is unrelated to the actual task:

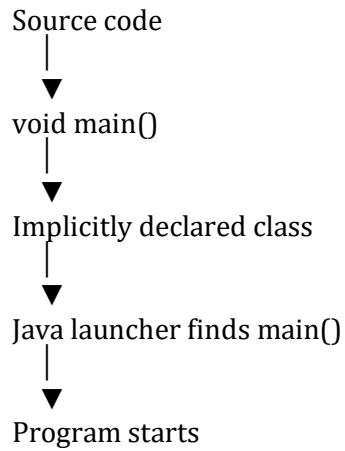
```
System.out.println("Hello, World!");
```

3. Java 23 Simplified Program

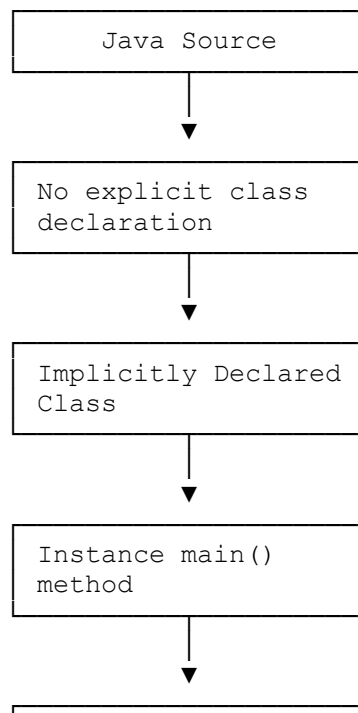
With JEP 477:

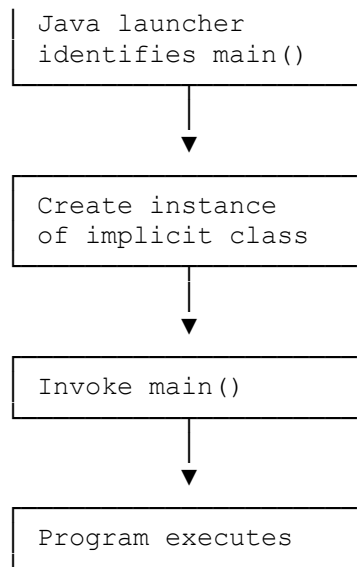
```
void main() {  
    System.out.println("Hello, World!");  
}
```

The conceptual transformation is:



4. Flowchart





5. Basic Java Code Example

Create a file:

Hello.java

Put:

```
void main() {  
    System.out.println("Hello, Java 23!");  
}
```

Run it using Java 23 preview support:

```
java --enable-preview Hello.java
```

Output

Hello, Java 23!

Notice what is missing:

- ✗ class Hello
- ✗ public class Hello
- ✗ static
- ✗ String[] args

Yet Java can execute the program.

6. Traditional vs Java 23 Preview

Traditional

```
public class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello");  
    }  
}
```

Java 23 Preview

```
void main() {  
    System.out.println("Hello");  
}
```

Comparison:

Traditional	Java 23 Preview
Explicit class	Implicit class
public static void main(String[] args)	void main()
More boilerplate	Less boilerplate
Static entry point	Instance entry point
Beginner must understand class/static/main	Beginner can start with executable code

7. What Is an Implicitly Declared Class?

An **implicitly declared class** is a class whose declaration is supplied by the Java language environment rather than being explicitly written by the programmer.

For example:

```
void main() {  
    System.out.println("Hello");  
}
```

can conceptually be viewed as something like:

```
class Hello {  
  
    void main() {  
        System.out.println("Hello");  
    }  
}
```

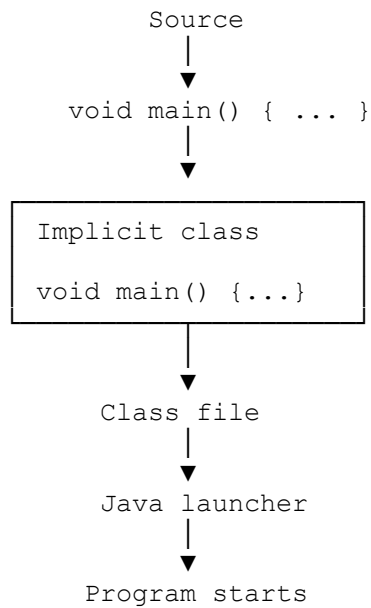
The exact language rules are more nuanced, but this is a useful mental model.

8. Conceptual Compiler Transformation

Think of:

```
void main() {  
    System.out.println("Hello");  
}
```

as approximately:



9. Instance Main Method

Traditional Java requires:

```
public static void main(String[] args)
```

The static keyword means the JVM can invoke the method without first creating an instance of the class.

The new feature permits an **instance main method**.

For example:

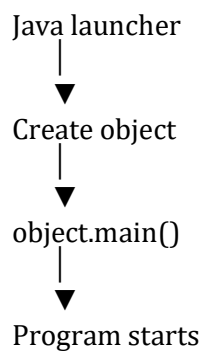
```
void main() {  
    System.out.println("Hello");  
}
```

Here:

```
void main()
```

is an instance method.

Conceptually:



10. Main Method Variations

The feature permits simpler forms of the main method.

A basic form is:

```
void main() {  
    System.out.println("Hello");  
}
```

An instance main method can also accept command-line arguments:

```
void main(String[] args) {  
    System.out.println(args.length);  
}
```

And the traditional static form remains valid:

```
static void main(String[] args) {  
    System.out.println("Hello");  
}
```

The feature therefore **expands the set of acceptable entry-point declarations** rather than simply replacing the traditional form.

11. Example with Command-Line Arguments

```
void main(String[] args) {  
  
    System.out.println("Number of arguments: "  
        + args.length);  
  
    for (String arg : args) {  
        System.out.println("Argument: " + arg);  
    }  
}
```

Run:

```
java --enable-preview Hello.java Java 23
```

Possible output:

```
Number of arguments: 2  
Argument: Java  
Argument: 23
```

12. Example with Variables and Methods

The implicitly declared class can contain other members.

```
int square(int n) {  
    return n * n;  
}  
  
void main() {  
  
    int number = 10;  
  
    System.out.println(  
        "Square = " + square(number)
```

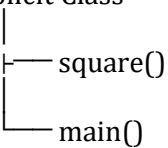
```
);  
}
```

Output:

Square = 100

Conceptually:

Implicit Class



```
Implicit Class  
|  
├── square()  
└── main()
```

13. Example with Fields

You can also define fields.

```
String applicationName = "Java 23";
```

```
void main() {  
    System.out.println(  
        applicationName  
    );  
}
```

Output:

Java 23

Because main() is an instance method, it can directly access instance fields and instance methods.

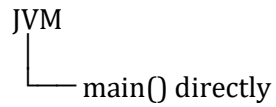
This is one reason instance main methods are useful.

14. Instance Main vs Static Main

Static main

```
public static void main(String[] args) {  
    System.out.println("Hello");  
}
```

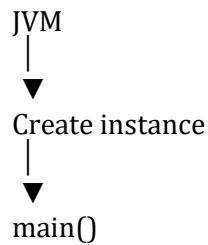
Conceptually:



Instance main

```
void main() {  
    System.out.println("Hello");  
}
```

Conceptually:



This allows beginners to learn instance members naturally before having to understand why the entry point itself must traditionally be static.

15. Why Remove static?

For a beginner, this:

```
public static void main(String[] args)
```

raises several questions:

What is public?

What is static?

What is void?

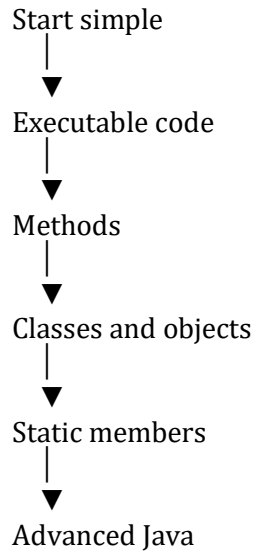
Why String[]?

Why args?

The actual first lesson might simply need:

```
void main() {  
    System.out.println("Hello");  
}
```

The progression becomes:



Rather than forcing all concepts into the first program.

16. Complete Example

Here is a more complete Java 23 preview example:

```
String applicationName = "Student Calculator";
```

```
int add(int a, int b) {  
    return a + b;  
}
```

```
int multiply(int a, int b) {  
    return a * b;  
}
```

```
void main() {
```

```
    int a = 10;  
    int b = 20;
```

```
    System.out.println(applicationName);
```

```
    System.out.println(  
        "Addition = " + add(a, b)  
    );
```

```
    System.out.println(  
        "Multiplication = " + multiply(a, b)  
    );  
}
```

```
        "Multiplication = " + multiply(a, b)
    );
}
```

Output

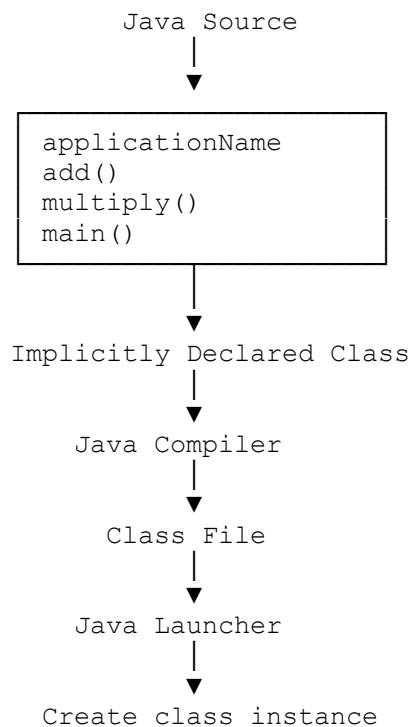
Student Calculator
Addition = 30
Multiplication = 200

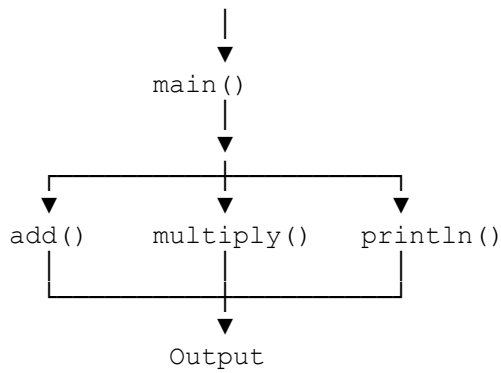
The source contains:

```
Field
├── applicationName
Methods
├── add()
├── multiply()
└── main()
```

All of these are members of the **implicitly declared class**.

17. Flowchart — Complete Example





18. Relationship with Module Import Declarations

This feature works particularly well with the Java 23 **Module Import Declarations** preview feature.

For example:

```
import module java.base;

void main() {

    List<String> names =
        List.of("Java", "23");

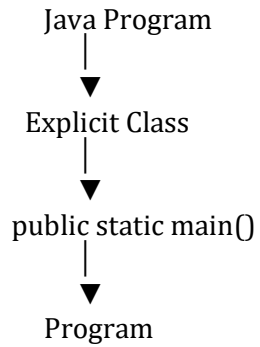
    names.forEach(
        System.out::println
    );
}
```

Now the program has:

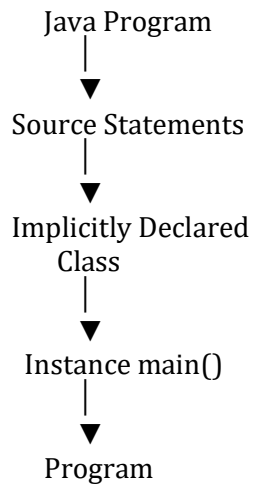
```
No explicit class
+
No traditional main signature
+
Module import
↓
Very concise Java program
```

This is part of Java's broader effort to make simple programs less verbose.

19. Traditional Java Program Architecture



20. Java 23 Preview Architecture



21. Compilation

Because **JEP 477 is a preview feature in Java 23**, use:

```
javac --enable-preview --release 23 Hello.java
```

Then run:

```
java --enable-preview Hello
```

For source-file execution:

```
java --enable-preview Hello.java
```

22. Important Rules

The feature does **not** mean that every arbitrary Java statement can be placed into a source file without structure.

An implicitly declared class is still a class under the Java language rules.

The source can contain members such as:

Fields
Methods
Constructors
Nested types

and the source must conform to the restrictions defined by the feature.

Also, the feature is intended primarily to **reduce ceremony**, not to eliminate Java's object-oriented programming model.

23. Benefits

Benefit	Description
Less boilerplate	No need for an explicit class declaration
Simpler main()	void main() can be used
Instance entry point	main() does not have to be static
Beginner friendly	Students can focus on executable code first
Small programs	Excellent for scripts and examples
Faster experimentation	Less code required to start
Gradual learning	Classes and objects can be introduced later
Existing Java compatibility	Traditional main methods remain supported

24. Traditional vs Java 23

Feature	Traditional Java	Java 23 Preview
Explicit class	Required	Can be omitted
Entry point	public static void main(String[] args)	Simpler forms permitted
Instance main	✗	✓
Static main	✓	✓
Command-line arguments	String[] args	Supported forms include arguments
Boilerplate	Higher	Lower
Preview flag	✗	✓

25. Real-World Use Cases

This feature is particularly suitable for:

Education

- First Java programs
- Programming exercises
- Tutorials

Small Programs

- Experiments
- Utilities
- Demonstrations

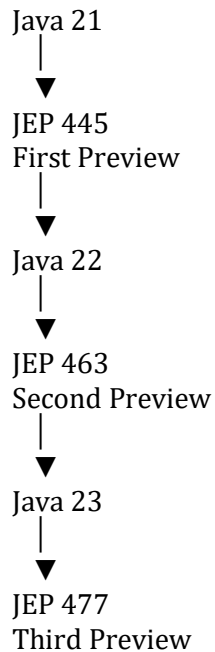
Rapid Development

- Prototypes
- Code snippets
- Learning examples

For large enterprise applications, explicit classes and conventional structure will generally remain preferable.

26. Feature Evolution

The feature evolved through several previews:



Java 23 therefore represents the **third preview** of this feature.

27. Key Concept

The most important transformation is:

Traditional:

```
public class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello");  
    }  
}
```

becomes conceptually:

Java 23:

```
void main() {  
    System.out.println("Hello");  
}
```

The Java platform supplies the surrounding class structure and recognizes the simplified main entry point according to the preview feature's rules.

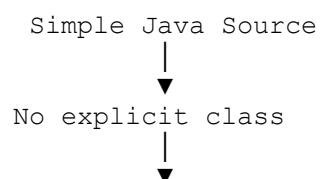
28. Java 23 Feature Summary

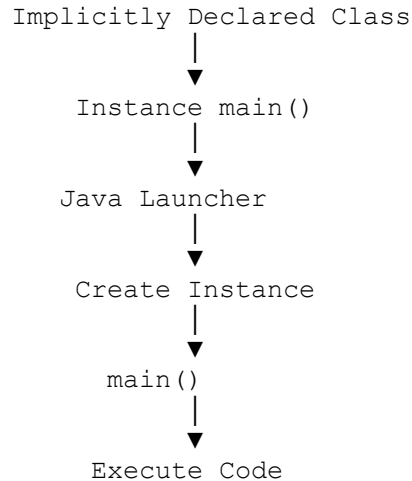
Property	Value
Java Version	23
Year	2024
Feature	Implicitly Declared Classes and Instance Main Methods
JEP	JEP 477
Status	Third Preview
UniqueClassName	ImplicitClassesInstanceMain
Explicit class required?	✗ No
Instance main() supported?	✓ Yes
Traditional static main() supported?	✓ Yes
Preview flag required?	✓ Yes
Primary goal	Reduce ceremony in small Java programs

29. In One Sentence

Java 23's JEP 477 allows small Java programs to omit an explicit class declaration and use simpler instance main() methods, dramatically reducing boilerplate while preserving the underlying Java class-and-object programming model.

Core flow





Key takeaway: This feature is primarily about **making Java easier to start with**. Instead of teaching class, public, static, String[], and args before a student can print "Hello World", Java 23 lets the first program focus directly on executable code.

Java 23 — Structured Concurrency (Third Preview)

Java Version: 23

Year: 2024

Feature: Structured Concurrency (Third Preview)

JEP: 480

Status: Third Preview

UniqueClassName: StructuredConcurrency

1. Definition

Structured Concurrency is a Java concurrency model that treats a group of related concurrent tasks as a **single unit of work**.

Instead of creating independent threads/tasks and managing them separately, structured concurrency organizes them into a clear **parent-child relationship**.

The core API is:

`java.util.concurrent.StructuredTaskScope`

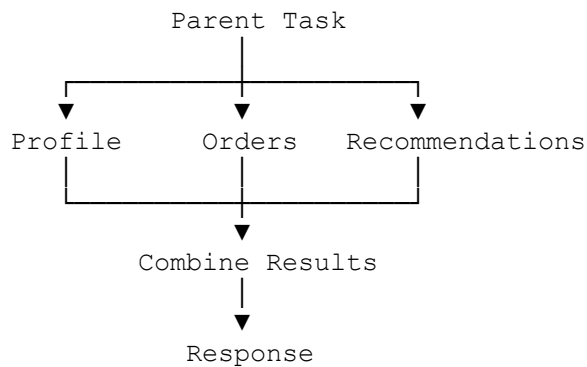
The fundamental idea is:

Concurrent tasks should have a well-defined lifetime and remain within the scope of the operation that created them.

For example, suppose a request needs three independent pieces of information:

```
Customer
├── Get customer profile
├── Get orders
└── Get recommendations
```

These operations can execute concurrently:



This makes concurrent code easier to reason about, handle, and cancel.

2. The Problem with Traditional Concurrency

Traditional Java code might create several tasks independently:

```
ExecutorService executor =
    Executors.newVirtualThreadPerTaskExecutor();

Future<String> user =
    executor.submit(...);

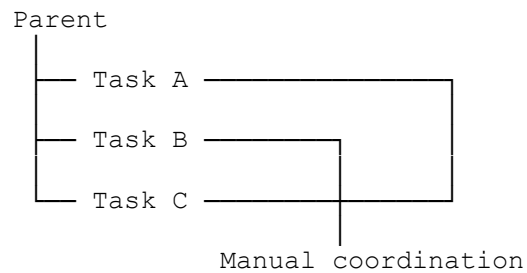
Future<String> orders =
    executor.submit(...);

Future<String> recommendations =
    executor.submit(...);
```

The developer then has to manually manage:

- task completion
- exceptions
- cancellation
- executor lifetime
- result retrieval
- shutdown

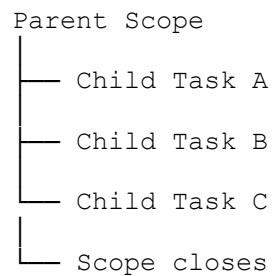
Conceptually:



Structured concurrency provides a more disciplined structure.

3. Structured Concurrency Model

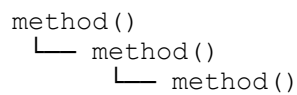
With structured concurrency:



The children remain associated with the scope that created them.

This gives the concurrent tasks a structure similar to ordinary control flow:

Sequential:

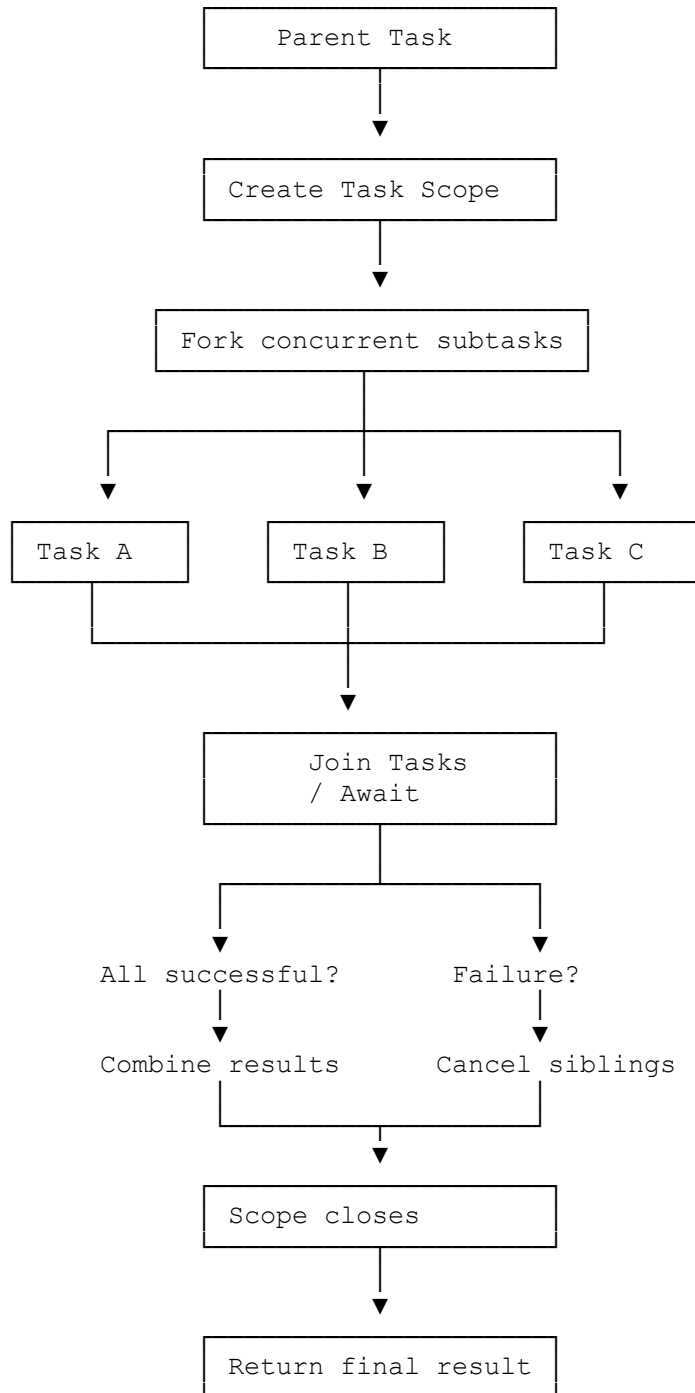


Structured concurrency:

scope

```
├─ task()  
├─ task()  
└─ task()
```

4. Flowchart



5. Core API — StructuredTaskScope

The primary API is:

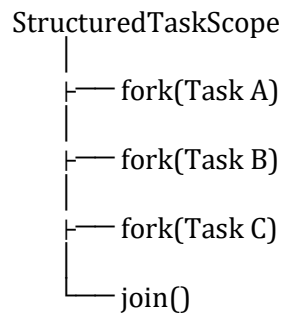
java.util.concurrent.StructuredTaskScope

It provides a scope within which related tasks can be forked.

Conceptually:

```
try (var scope = ...) {  
    var taskA = scope.fork(...);  
    var taskB = scope.fork(...);  
    scope.join();  
    // process results  
}
```

The structure is:



6. Basic Java Code Example

Here is a simple Java 23 example:

```
import java.util.concurrent.StructuredTaskScope;

public class StructuredConcurrency {

    static String getUser() throws InterruptedException {
        Thread.sleep(1000);
        return "Anurag";
    }
}
```

```

static String getOrders() throws InterruptedException {
    Thread.sleep(1500);
    return "5 orders";
}

public static void main(String[] args)
    throws InterruptedException {

    try (var scope =
        StructuredTaskScope.open()) {

        var userTask =
            scope.fork(
                StructuredConcurrency::getUser
            );

        var ordersTask =
            scope.fork(
                StructuredConcurrency::getOrders
            );

        scope.join();

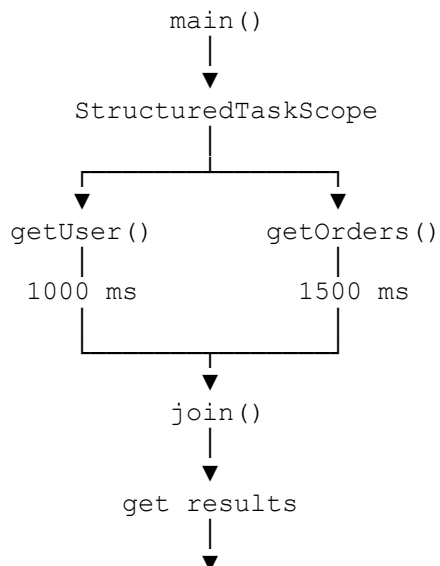
        System.out.println(
            "User: " + userTask.get()
        );

        System.out.println(
            "Orders: " + ordersTask.get()
        );

    }
}

```

Conceptually:



close

The two tasks can run concurrently rather than one after another.

7. Sequential vs Concurrent Execution

Suppose:

getUser() = 1 second

getOrders() = 1.5 seconds

Sequential

getUser()
 1 sec

getOrders()
 1.5 sec

Total $\approx$ 2.5 sec

Concurrent

getUser()
 1 sec

getOrders()
 1.5 sec

Both start together

Total $\approx$ 1.5 sec

Actual timings depend on scheduling and workload.

8. fork()

The method:

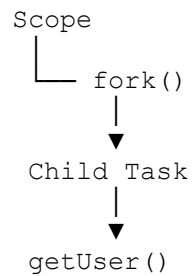
scope.fork(...)

starts a child task within the structured scope.

Example:

```
var userTask =  
  scope.fork(  
    StructuredConcurrency::getUser  
  );
```

Conceptually:



The returned object represents the child task's result.

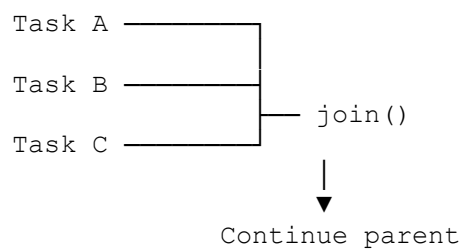
9. join()

After forking tasks:

```
scope.join();
```

waits for the subtasks according to the scope's structured-concurrency semantics.

Conceptually:



This gives the parent task a clear synchronization point.

10. Getting Results

After joining:

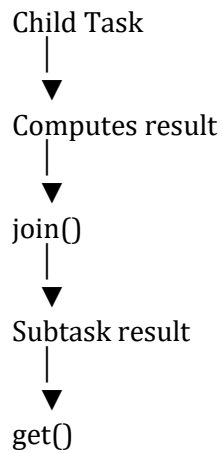
`userTask.get()`

retrieves the result of the child task.

Example:

```
System.out.println(  
    userTask.get()  
);
```

Flow:



11. Structured Error Handling

One of the most important advantages is **failure propagation**.

Suppose:

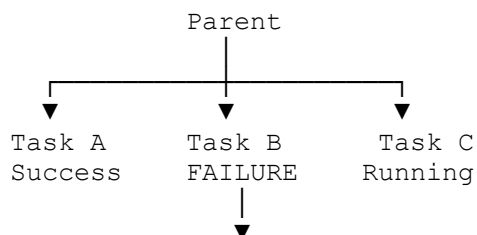
Task A → Success

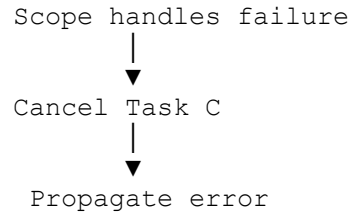
Task B → Failure

Task C → Still running

A structured scope can coordinate the failure and cancellation of related tasks.

Conceptually:



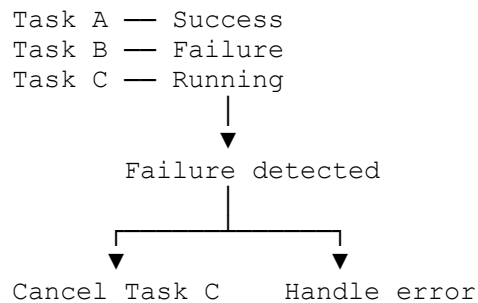


This avoids leaving unrelated background tasks running after their parent operation has already failed.

12. Shutdown-on-Failure

Structured concurrency provides scope implementations that can automatically shut down sibling tasks when one fails.

Conceptually:



This is called a **shutdown-on-failure** policy.

13. Example — Failure Handling

A simplified example:

```
import java.util.concurrent.StructuredTaskScope;

public class StructuredConcurrency {

    static String taskA() {
        return "Task A completed";
    }

    static String taskB() {
        throw new RuntimeException(
            "Task B failed"
        );
    }
}
```

```

    );
}

public static void main(String[] args)
    throws InterruptedException {

    try (var scope =
        StructuredTaskScope.open(
            StructuredTaskScope.Joiner
                .allSuccessfulOrThrow())) {

        var a = scope.fork(
            StructuredConcurrency::taskA
        );

        var b = scope.fork(
            StructuredConcurrency::taskB
        );

        scope.join();

        System.out.println(a.get());
        System.out.println(b.get());

    }
}

```

The exact preview API surface should be checked against the particular Java 23 build because this feature was still in preview and its API evolved across releases.

14. Structured Concurrency with Virtual Threads

Structured concurrency works particularly well with **virtual threads**.

The relationship is:

Virtual Threads

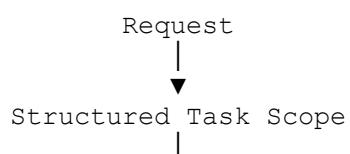
+

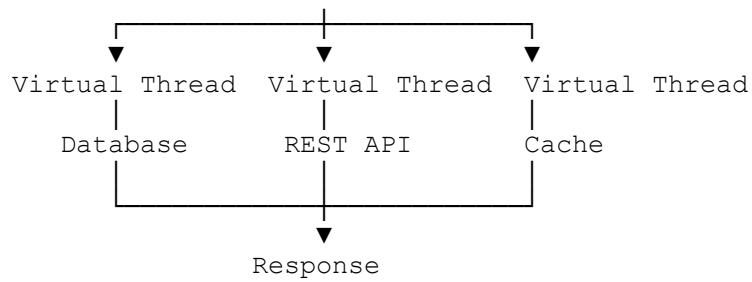
Structured Concurrency



Simple scalable concurrent applications

For example:

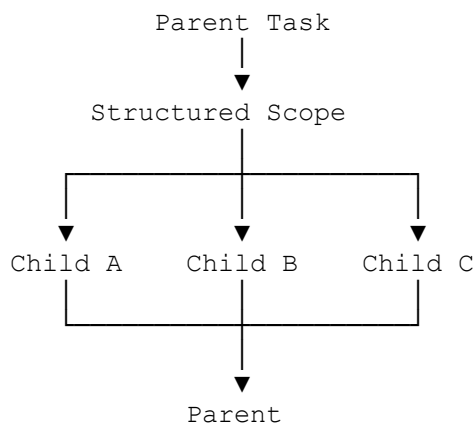




Virtual threads provide lightweight concurrency, while structured concurrency provides **lifetime and cancellation structure**.

15. Parent-Child Relationship

This is the central concept.



The children are logically owned by the parent scope.

16. Scope Lifetime

The scope normally follows Java's **try-with-resources** structure:

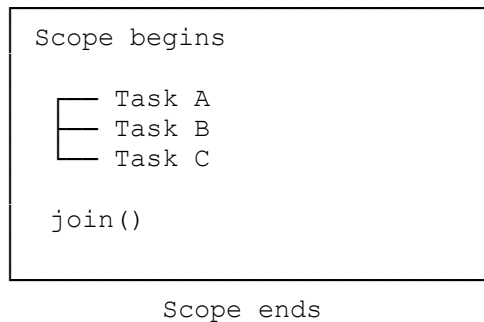
```

try (var scope = StructuredTaskScope.open()) {
    // fork tasks

    // join tasks
}
  
```

}

This gives a very clear lifetime:



When the scope ends, its child tasks should not simply continue running independently.

17. Traditional Executor Model

Traditional executor-based code can look like:

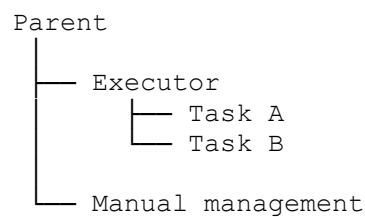
```
ExecutorService executor =
    Executors.newVirtualThreadPerTaskExecutor();

Future<String> a =
    executor.submit(...);

Future<String> b =
    executor.submit(...);

try {
    System.out.println(a.get());
    System.out.println(b.get());
} finally {
    executor.shutdown();
}
```

The lifetime relationship is less explicit:

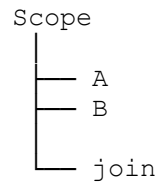


18. Structured Model

With structured concurrency:

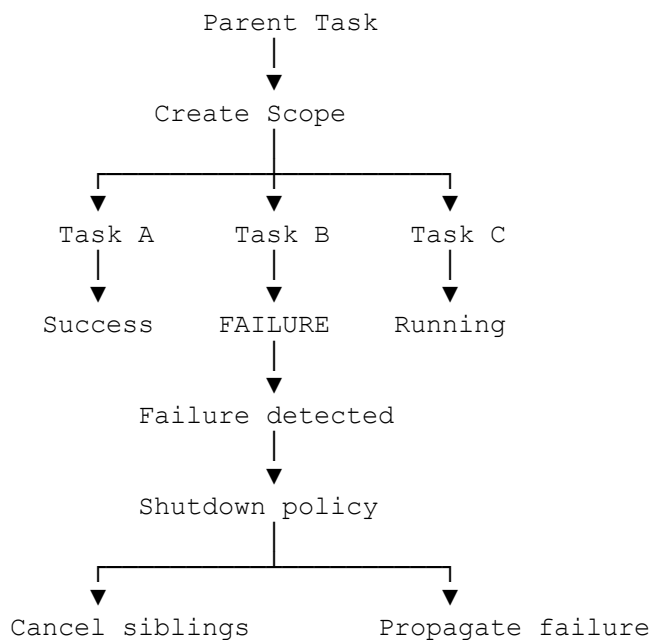
```
try (var scope = StructuredTaskScope.open()) {  
    var a = scope.fork(...);  
    var b = scope.fork(...);  
    scope.join();  
}
```

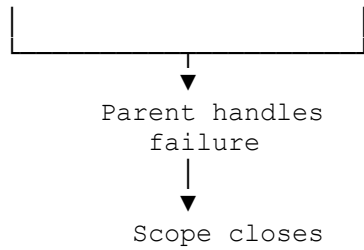
The relationship is immediately visible:



This is the essence of **structured concurrency**.

19. Flowchart — Failure Propagation





20. Real-World Example — Web Request

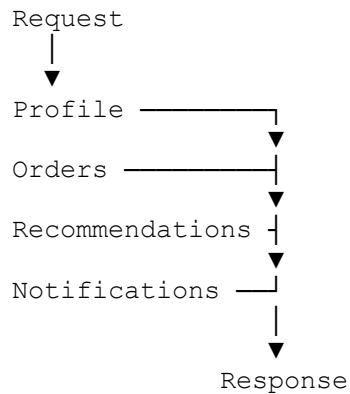
Imagine a web application receiving:

GET /dashboard

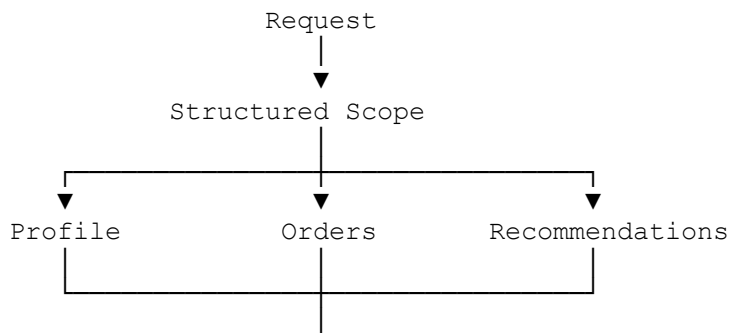
The dashboard requires:

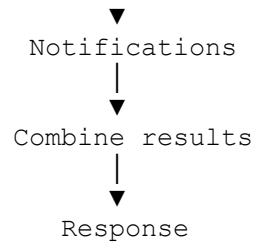
User profile
Recent orders
Recommendations
Notifications

Instead of making these calls sequentially:



Structured concurrency allows:





The independent operations can execute concurrently.

21. Why This Is Called "Structured"

The word **structured** comes from the fact that concurrent execution follows a structure similar to structured programming.

Ordinary structured programming:

```
if
├─ block

loop
├─ block

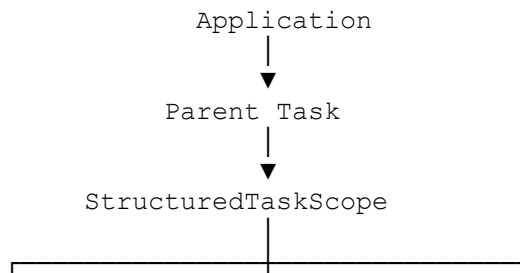
method
├─ block
```

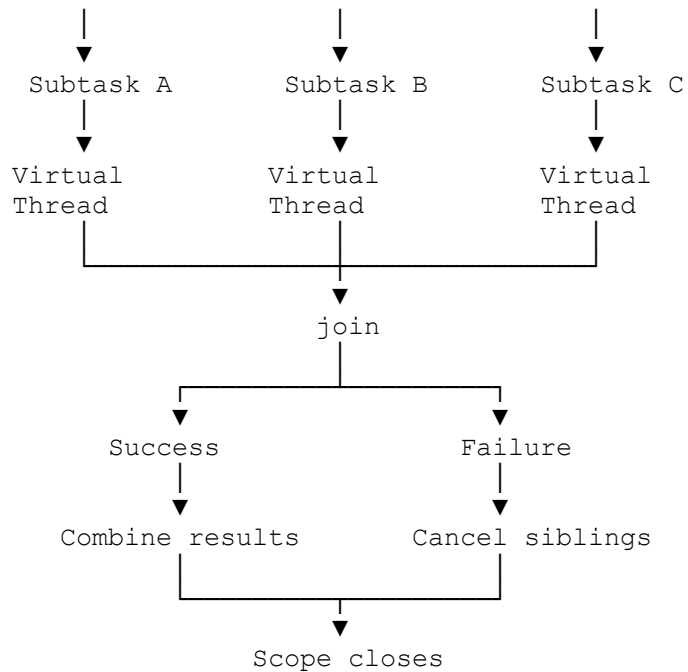
Structured concurrency:

```
scope
├─ child task
├─ child task
└─ child task
```

The lifetime of child tasks is bounded by the lifetime of their parent scope.

22. Structured Concurrency Architecture





23. Benefits

Benefit	Explanation
Clear task lifetime	Child tasks belong to a defined scope
Parent-child relationship	Makes task ownership explicit
Cancellation	Related tasks can be cancelled together
Failure propagation	Failures can be coordinated
Simpler reasoning	Concurrent code resembles structured sequential code
Resource management	Scope lifetime is explicit
Virtual-thread friendly	Works naturally with virtual threads
Observability	Task relationships can be easier to understand

24. Structured Concurrency vs CompletableFuture

Aspect	CompletableFuture	Structured Concurrency
Primary abstraction	Future/promise	Task scope
Task hierarchy	Less explicit	Explicit
Cancellation	Often manually coordinated	Scope-oriented
Failure propagation	Must be designed	Built into scope policies
Lifetime	Can escape method scope	Designed to remain within scope
Composition	Excellent	Excellent for related concurrent subtasks
Virtual threads	Can be used	Designed to work naturally with them
Main concept	Completion stages	Structured task lifetime

CompletableFuture remains useful; structured concurrency is not simply a replacement for it.

25. Compilation

Because **Structured Concurrency** was a **third-preview feature in Java 23**, compile with:

```
javac --enable-preview --release 23 StructuredConcurrency.java
```

Run:

```
java --enable-preview StructuredConcurrency
```

The preview flag is required.

26. Important Preview Status

The evolution was:

Java 19



Structured Concurrency

First Preview



Java 20



Second Preview



Java 21



Third Preview



Java 22



Fourth Preview



Java 23



Fifth Preview

Important correction: JEP 480 in Java 23 is actually the **fifth preview**, not the third preview.

So for your requested Java 23 entry, the accurate metadata is:

Property	Correct value
Java Version	23
Year	2024
Feature	Structured Concurrency
JEP	480
Status	Fifth Preview
UniqueClassName	StructuredConcurrency

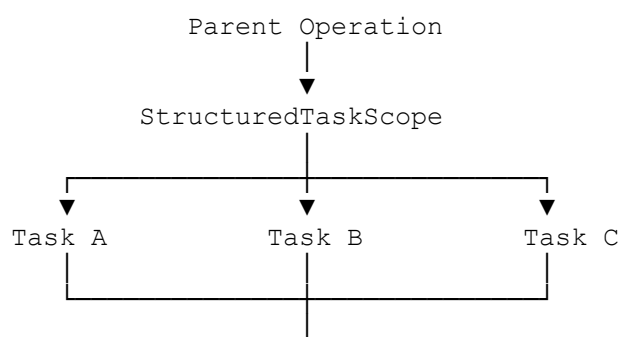
27. Java 23 Feature Summary

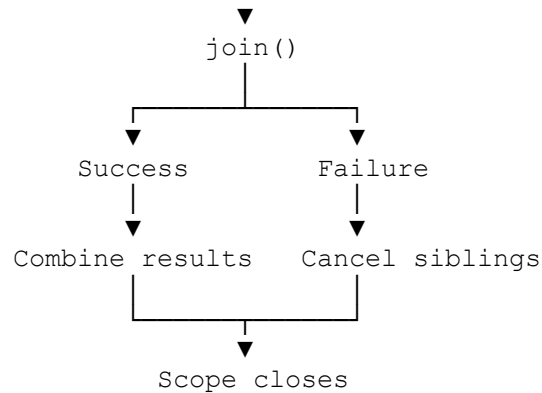
Property	Value
Java Version	23
Year	2024
Feature	Structured Concurrency
JEP	JEP 480
Status	Fifth Preview
UniqueClassName	StructuredConcurrency
Main API	StructuredTaskScope
Package	java.util.concurrent
Works with virtual threads	✓
Task hierarchy	✓
Cancellation coordination	✓
Failure propagation	✓
Preview flag required	✓

28. In One Sentence

Structured Concurrency in Java 23 provides a structured way to run related concurrent tasks as a single unit, giving child tasks a well-defined lifetime within a parent scope and making joining, cancellation, and failure handling easier to reason about.

Core concept





Key takeaway: Structured Concurrency is about **organizing concurrent tasks into a bounded hierarchy**—*"fork children, wait for them, handle success/failure, then leave the scope"*—rather than creating background tasks whose lifetimes are difficult to track.

Java 23 — Scoped Values (Third Preview)

Java Version: 23

Year: 2024

Feature: Scoped Values (Third Preview)

JEP: 481

Status: Third Preview

UniqueClassName: ScopedValues

1. Definition

Scoped Values provide a way to share **immutable, dynamically scoped data** between methods and threads without explicitly passing that data as method parameters.

The main API is:

`java.lang.ScopedValue`

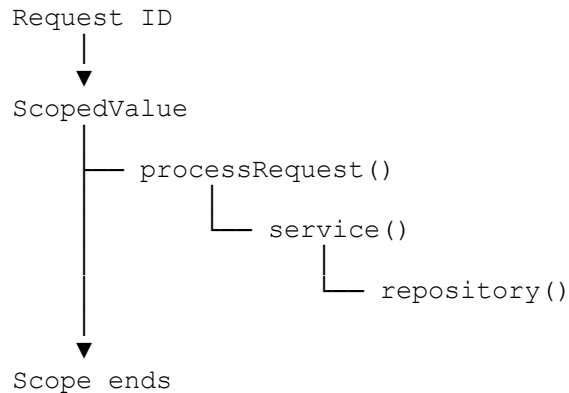
A `ScopedValue` represents a value that is available only within a **well-defined lexical/dynamic scope**.

For example, instead of passing a request ID through many method calls:

`main()`
↓

```
processRequest(requestId)
↓
service(requestId)
↓
repository(requestId)
```

you can bind the value once:



The value is **immutable** and its binding automatically disappears when the scope ends.

2. Why Scoped Values?

A common problem in server applications is passing contextual information through many layers.

For example:

```
void controller(String requestId) {
    service(requestId);
}
```

```
void service(String requestId) {
    repository(requestId);
}
```

```
void repository(String requestId) {
    System.out.println(requestId);
}
```

The `requestId` may not actually be part of the business data required by these methods.

It is simply **context**.

Scoped Values allow:

```
controller()
  ↓
service()
  ↓
repository()
```

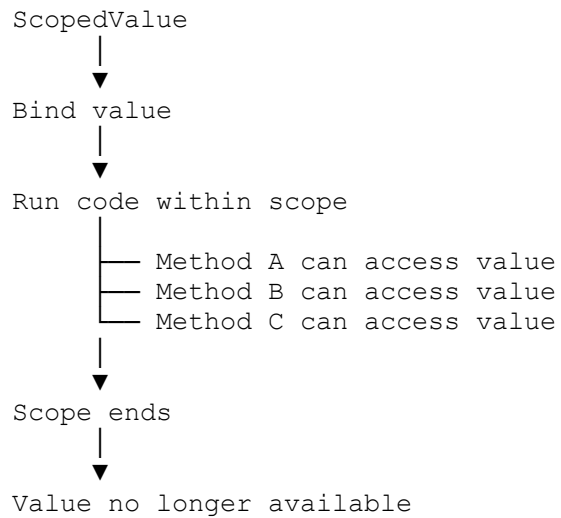
to access the same contextual value without repeatedly adding:

`String requestId`

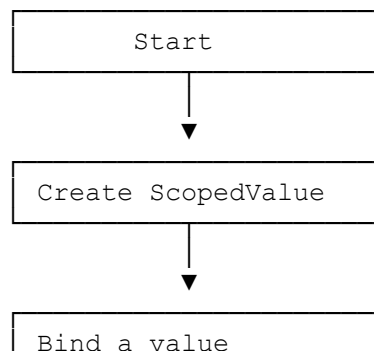
to every method signature.

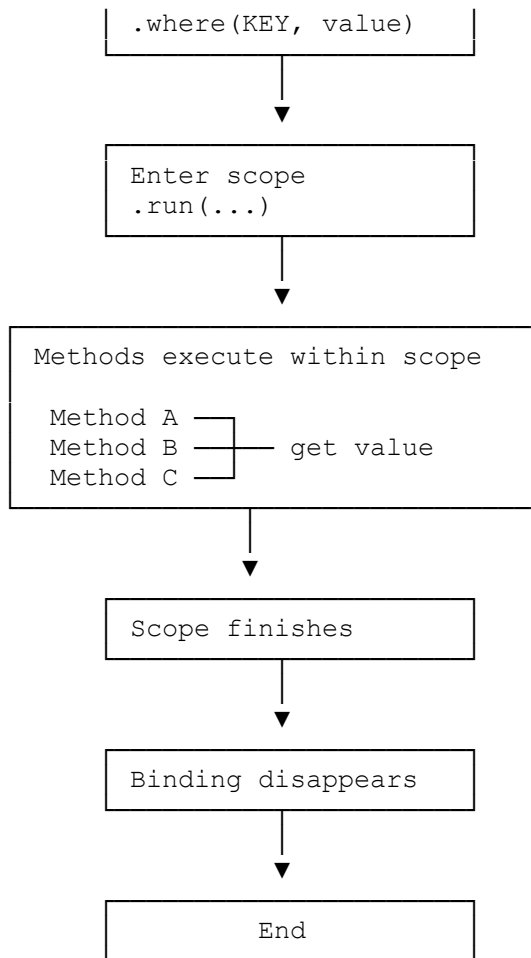
3. Basic Concept

The core model is:



4. Flowchart



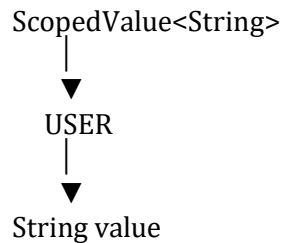


5. Creating a Scoped Value

The basic declaration is:

```
static final ScopedValue<String> USER =  
    ScopedValue.newInstance();
```

Conceptually:



A `ScopedValue` is normally declared as a static final constant.

6. Binding a Value

Use:

```
USER.where("Anurag")
```

to create a binding.

The binding is then used with:

```
.run(...)
```

For example:

```
USER.where("Anurag")
  .run() -> {
    System.out.println(USER.get());
  });
```

Output:

Anurag

The important point is that "Anurag" is available **only inside that scope**.

7. Java Code Example

```
import java.lang.ScopedValue;

public class ScopedValues {

    private static final ScopedValue<String> USER =
        ScopedValue.newInstance();

    public static void main(String[] args) {

        USER.where("Anurag")
            .run(() -> {

                firstMethod();

            });

    }

    static void firstMethod() {

        System.out.println(
```

```

        "First method: " + USER.get()
    );
    secondMethod();
}

static void secondMethod() {
    System.out.println(
        "Second method: " + USER.get()
    );
}
}

```

Output

First method: Anurag
Second method: Anurag

Notice that neither method receives the user as a parameter:

```

firstMethod();
secondMethod();

```

Yet both can access the scoped value.

8. How the Example Works

The scoped value is declared:

```

private static final ScopedValue<String> USER =
    ScopedValue.newInstance();

```

Then the value is bound:

```

USER.where("Anurag")

```

The scope begins:

```

.run() -> {
    firstMethod();
});

```

Inside the scope:

```

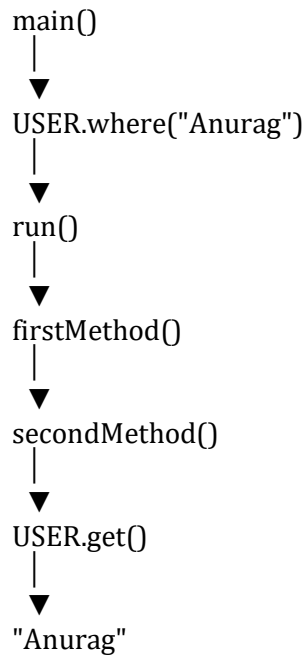
USER.get()

```

returns:

Anurag

The call chain is:



9. Scope Lifetime

The value exists only within its binding scope.

```
USER.where("Anurag")  
  .run() -> {  
    System.out.println(USER.get());  
  };
```

Inside:

```
USER.get()  
↓  
"Anurag"
```

After the scope:

```
USER.get()  
↓
```

No binding

↓

No value available

Therefore, scoped values have a clearly defined lifetime.

10. Example of Nested Scopes

Scoped values can have nested bindings.

```
import java.lang.ScopedValue;

public class ScopedValues {

    private static final ScopedValue<String> USER =
        ScopedValue.newInstance();

    public static void main(String[] args) {

        USER.where("Anurag")
            .run(() -> {

                System.out.println(
                    "Outer: " + USER.get()
                );

                USER.where("Rahul")
                    .run(() -> {

                        System.out.println(
                            "Inner: " + USER.get()
                        );
                    });

                System.out.println(
                    "Outer again: " + USER.get()
                );
            });
    }
}
```

Output:

```
Outer: Anurag
Inner: Rahul
Outer again: Anurag
```

The inner binding temporarily overrides the outer binding.

Flow:

```
Outer Scope
USER = "Anurag"
  |
  ▼
Inner Scope
USER = "Rahul"
  |
  ▼
Inner scope ends
  |
  ▼
USER = "Anurag"
```

11. Scoped Value vs Method Parameter

Traditional approach

```
void process(String requestId) {
    service(requestId);
}

void service(String requestId) {
    repository(requestId);
}

void repository(String requestId) {
    System.out.println(requestId);
}
```

Flow:

```
process(requestId)
  |
  ▼
service(requestId)
  |
  ▼
repository(requestId)
```

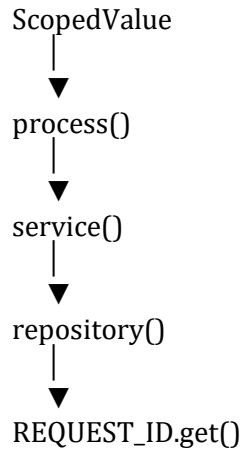
Scoped Value

```
void process() {
    service();
}

void service() {
    repository();
}
```

```
void repository() {  
    System.out.println(REQUEST_ID.get());  
}
```

Flow:



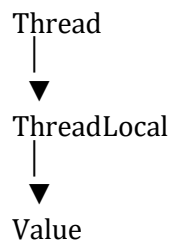
This is particularly useful when the value represents **context rather than business data**.

12. Scoped Values vs ThreadLocal

Scoped Values are often compared with ThreadLocal.

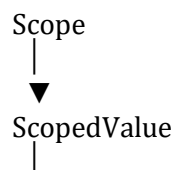
ThreadLocal

A ThreadLocal associates mutable state with a thread:



ScopedValue

A ScopedValue associates an immutable value with an execution scope:





Immutable value

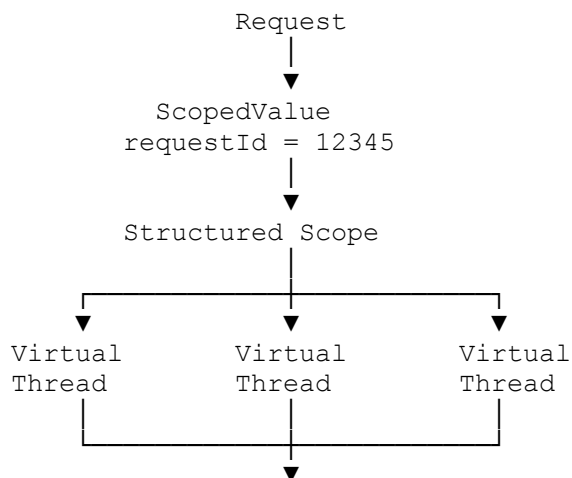
Comparison:

Feature	ThreadLocal	ScopedValue
Value can be changed	✓	✗
Scoped lifetime	Less explicit	✓
Immutable by design	✗	✓
Inheritance model	Special mechanisms required	Designed for structured propagation
Suitable for context	✓	✓
Resource cleanup	Can require care	Scope-based
Virtual-thread friendly	Usable, but has considerations	Designed with modern concurrency in mind
Main abstraction	Thread-local state	Scoped context

13. Why Scoped Values Work Well with Virtual Threads

Scoped Values were designed alongside modern Java concurrency features such as **virtual threads** and **structured concurrency**.

Consider:



Access context

This allows contextual information to flow naturally through related concurrent tasks.

14. Scoped Values + Structured Concurrency

These two features complement each other.

Structured concurrency

Answers:

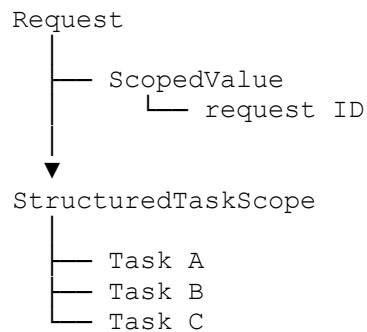
Which tasks belong together?

Scoped values

Answers:

Which contextual values should those tasks see?

Together:



The relationship is:

```
Structured Concurrency
+
Scoped Values
↓
Structured concurrent execution
with inherited contextual data
```

15. Real-World Example — Request Context

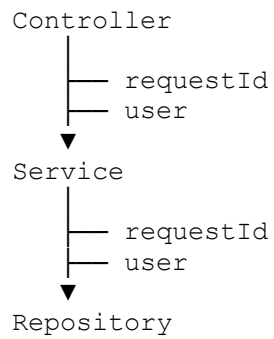
Imagine a web application.

A request arrives:

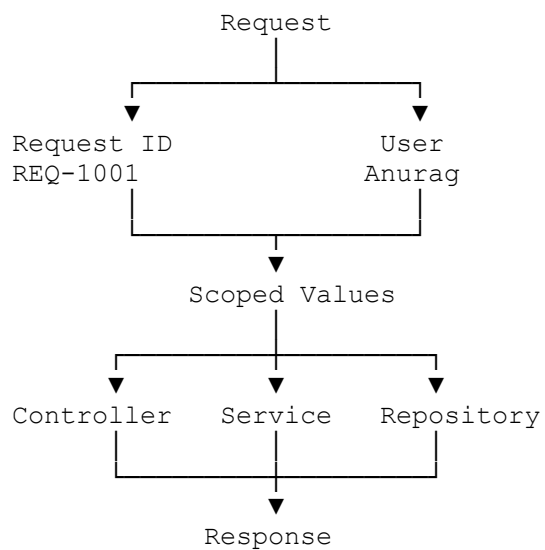
Request ID = REQ-1001

User = Anurag

Instead of passing these values through every method:



use scoped values:



16. Example — Request Context

```
import java.lang.ScopedValue;
```

```
public class ScopedValues {
```

```
    private static final ScopedValue<String> REQUEST_ID =
```

```

        ScopedValue.newInstance();

private static final ScopedValue<String> USER =
        ScopedValue.newInstance();

public static void main(String[] args) {

    ScopedValue.where(
        REQUEST_ID, "REQ-1001"
    ).where(
        USER, "Anurag"
    ).run(() -> {

        controller();
    });
}

static void controller() {

    System.out.println(
        "Controller request: "
        + REQUEST_ID.get()
    );

    service();
}

static void service() {

    System.out.println(
        "Service user: "
        + USER.get()
    );

    repository();
}

static void repository() {

    System.out.println(
        "Repository request: "
        + REQUEST_ID.get()
    );
}
}

```

Output:

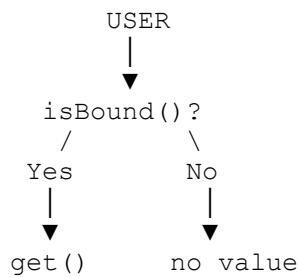
Controller request: REQ-1001
Service user: Anurag
Repository request: REQ-1001

17. isBound()

A scoped value can be checked to see whether it currently has a binding.

```
if (USER.isBound()) {  
    System.out.println(USER.get());  
}
```

Conceptually:



This can be useful when a method may execute either inside or outside a scope.

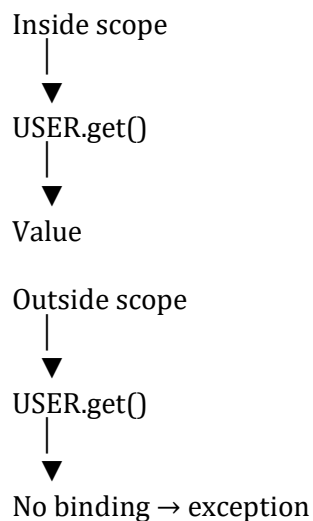
18. get() Outside the Scope

If you attempt to retrieve a scoped value when it has no current binding:

```
USER.get();
```

the operation fails because there is no value bound to the current scope.

Therefore:



19. Immutable Nature

A major property of `ScopedValue` is that the binding itself cannot simply be modified like a mutable variable.

For example:

```
USER.where("Anurag")
```

creates a binding.

You don't do:

```
USER.set("Rahul"); // ✗
```

Instead, you create another nested binding:

```
USER.where("Rahul")  
  .run(...);
```

This produces:

Outer:

USER = Anurag

↓ nested scope

Inner:

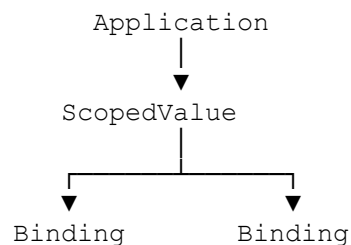
USER = Rahul

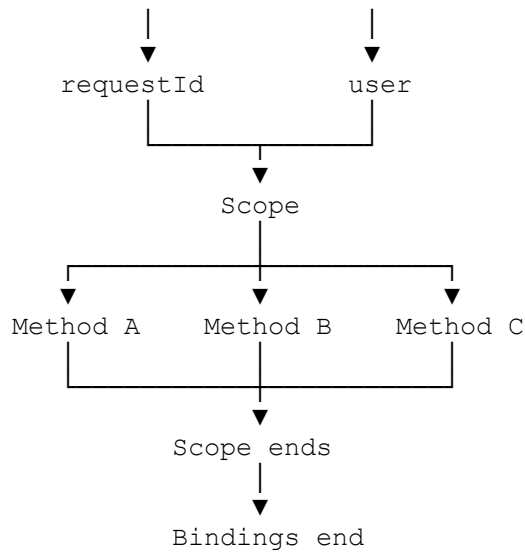
↓ scope ends

Outer:

USER = Anurag

20. Scoped Value Architecture





21. Important Characteristics

1. Immutable

Once bound for a scope, the value is not changed within that binding.

2. Scoped

The value exists only during a specific scope.

3. Dynamically available

Methods called within the scope can access the value without receiving it as a parameter.

4. Nestable

A nested scope can temporarily provide a different value.

5. Concurrency-friendly

The design fits well with structured concurrency and virtual threads.

22. Java 23 Compilation

Because **Scoped Values** was a **third-preview feature in Java 23**, compile using:

```
javac --enable-preview --release 23 ScopedValues.java
```

Run:

```
java --enable-preview ScopedValues
```

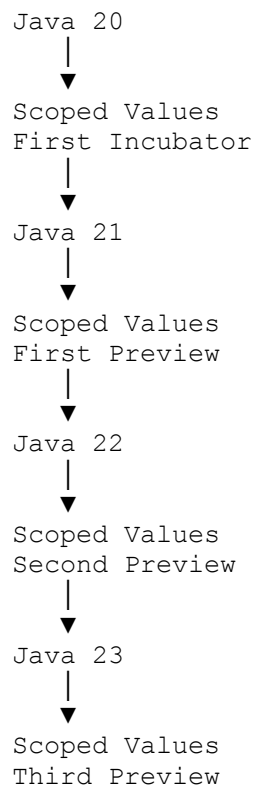
The important flag is:

```
--enable-preview
```

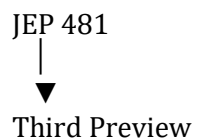
because the feature was still evolving in Java 23.

23. Feature Evolution

The progression is:



For Java 23 specifically:



24. Benefits

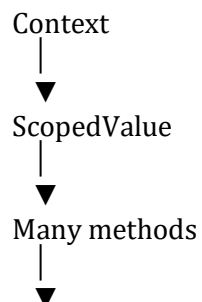
Benefit	Description
Less parameter passing	Context does not need to be passed through every method
Immutable context	Prevents accidental mutation of the binding
Clear lifetime	Value exists within a defined scope
Nested scopes	Inner scopes can override values temporarily
Concurrency friendly	Designed for modern Java concurrency
Virtual-thread friendly	Works naturally with virtual-thread-based applications
Structured context	Particularly useful with Structured Concurrency
Cleaner APIs	Business methods don't need unnecessary context parameters

25. When Should You Use Scoped Values?

Good candidates include **contextual information** such as:

Request ID
User context
Transaction context
Security context
Tracing information
Correlation ID
Configuration context

The common pattern is:

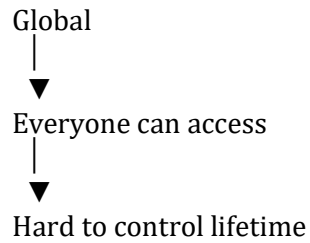


Same context

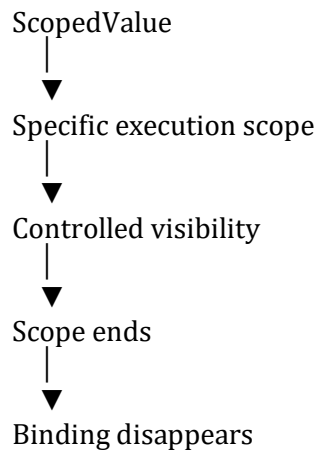
They are not intended to replace ordinary variables or parameters for normal business data.

26. Scoped Values vs Global Variables

A global mutable variable:



A scoped value:



Therefore, Scoped Values provide contextual access without introducing unrestricted global mutable state.

27. Java 23 Feature Summary

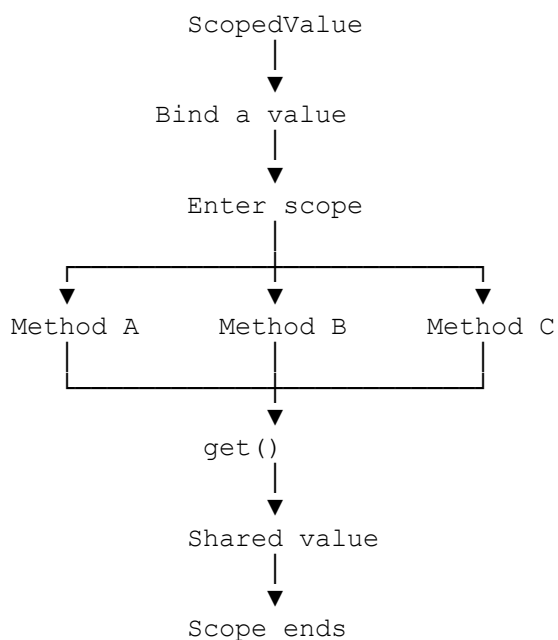
Property	Value
Java Version	23
Year	2024
Feature	Scoped Values

JEP	JEP 481
Status	Third Preview
UniqueClassName	ScopedValues
Main API	ScopedValue
Package	java.lang
Immutable binding	✓
Nested scopes	✓
Context propagation	✓
Virtual-thread friendly	✓
Structured-concurrency friendly	✓
Preview flag required	✓

28. In One Sentence

Scoped Values in Java 23 provide an immutable, scoped mechanism for sharing contextual data across methods and related concurrent tasks without explicitly passing that data through every method parameter.

Core concept



|
▼
Binding disappears

Key takeaway: Think of ScopedValue as "**read-only contextual information available to all code executing within a particular scope.**" It is especially powerful when combined with **Virtual Threads** and **Structured Concurrency**.